

Contents

Contents					
1	Template	2			
2	Data Structures	2			
2.1	Segment Tree	2			
2.2	Merge Sort Tree	2			
2.3	Segment Tree Recursive	2			
2.4	Lazy	2			
2.5	Persistent Segment Tree	3			
2.6	Dynamic Segment Tree	3			
2.7	Sparse Table	3			
2.8	Treap	3			
2.9	BIT	5			
2.10	DSU	5			
2.11	DSU with Rollback	5			
2.12	Dynamic Connectivity	6			
2.13	Trie	6			
2.14	Mo's Algorithm	6			
3	Graphs	6			
3.1	2-SAT	6			
3.2	Block Cut Tree	7			
3.3	Bellman-Ford Cycle	7			
3.4	Bellman-Ford Path	7			
3.5	Bipartite Check	7			
3.6	Bridges	7			
3.7	Bridge Tree	8			
3.8	Dijkstra	8			
3.9	Dinic	8			
3.10	Euler Path Directed	9			
3.11	Euler Path Undirected	9			
3.12	Flood Fill	9			
3.13	Floyd-Warshall	9			
3.14	Kruskal	9			
3.15	Maximum Clique	9			
3.16	MCMF	10			
3.17	Prims	10			
3.18	SCC Kosaraju	10			
3.19	State Space Graph	11			
3.20	Topological Sort	11			
3.21	Virtual Tree	11			
4	Trees	11			
4.1	Centroid Decomposition	11			
4.2	Tree Diameter	11			
4.3	Euler Tour	11			
4.4	Heavy Light Decomposition	12			
4.5	LCA	12			
4.6	Min/Max Weight on Path	12			
4.7	Intersection Path	12			
4.8	Union Path	13			
4.9	Prufer Code Build	13			
4.10	Prufer Code Restore	13			
4.11	Tree Rerooting	13			
4.12	Tree Matching	14			
4.13	Rooted Tree Hashing	14			
4.14	Tree Isomorphism	14			
5	Number Theory	14			
5.1	Euler Totient	14			
5.2	Mobius Function	14			
5.3	Chinese Remainder Theorem	14			
5.4	Binpow	14			
6	Combinatorics	14			
6.1	nCr	14			
7	Math				14
7.1	FFT	14			
7.2	FWDT	15			
7.3	NTT	15			
7.4	FFT Any Mod	15			
7.5	Or Convolution	15			
7.6	Xor Basis	15			
7.7	Matrix	16			
8	Strings				16
8.1	Phi Function	16			
8.2	Z Function	16			
8.3	Aho Corasick	16			
8.4	Dynamic Aho Corasick	17			
8.5	Suffix Array	17			
8.6	Hashing	17			
8.7	Palindromic Tree	17			
8.8	Manacher	17			
9	Dp				18
9.1	Convex Hull Trick	18			
9.2	Knuth Division	18			
9.3	Li Chao Tree	18			
10	Geometry 2D				18
10.1	Point	18			
10.2	Line	19			
10.3	Circle	20			
10.4	Tangent	20			
10.5	Convex Hull	21			
10.6	Polygon	21			
11	Miscellaneous				21
11.1	Rnd template	21			

1 Template

```

1 // :<,'>w !sed 's/\\/.*/' | tr -d '[:space:]' | tr A-Z a-z |
  md5sum | cut -c6
2 //echo "vnoemap <space>h :w !sed 's/\\/.*/' | tr -d '[:space:]'
  | tr A-Z a-z | md5sum | cut -c6<CR>" >> ~/.vimrc
3 #include <bits/stdc++.h>
4
5 using namespace std;
6
7 #define all(v) v.begin(),v.end()
8 #define rall(v) v.rbegin(),v.rend()
9 #define pb push_back
10 #define sz(a) int(a.size())
11 #define F first
12 #define S second
13 typedef long long ll;
14 typedef vector<int> vi;
15 typedef vector<ll> vll;
16 typedef pair<int,int> pii;
17 typedef pair<ll,ll> pll;
18 const int MOD = 1e9+7;
19 const int INF = INT_MAX;
20 const ll INF64 = LLONG_MAX;
21 const long double EPS = 1e-9;
22 const long double PI = acos(-1.0L);
23 void setIO(string p){
24     freopen((p + ".in").c_str(), "r", stdin);
25     freopen((p + ".out").c_str(), "w", stdout);
26 }
27
28 void solve(){
29
30 }
31
32
33 int main(){
34     ios_base::sync_with_stdio(0);
35     cin.tie(0);
36     cout.tie(0);
37     int tc = 1;
38     //cin >> tc;
39     for (int t = 1; t <= tc; t++)solve();
40 }

```

2 Data Structures

2.1 Segment Tree

```

1 // TOTAL: c88465
2 struct Node{
3     ll val = 0;
4 }; // #6368df
5 Node operator+(Node a,Node b){return {a.val+b.val};}
6 class Segment_Tree{
7 public:
8     int n;
9     vector<Node> tree;
10
11     Segment_Tree(int x = 1e5+10){
12         n = x;
13         tree.resize(2*n+1);
14     } // #9ba7b9
15
16     void update(int p, ll val){
17         for(tree[p+=n] = {val};p>1;p>=1)
18             tree[p>>1] = tree[p]+tree[p^1];
19         //for (tree[p+=n] = {val};p>= 1;)

```

```

20         // t[p] = t[p<<1]+t[p<<1|1]; // #9c6ca8
21     } // #fd122a
22 ll query(int l,int r){
23     Node s;
24     for(l+=n,r+=n ; l<r ; l>=1 , r>=1){
25         if(l&1)s = s+ tree[l++];
26         if(r&1)s = s+ tree[--r];
27     }
28     return s.val;
29 } // #409731
30 }; // #19ff3f

```

2.2 Merge Sort Tree

```

1 // Hash: #545fdb
2 struct Node{
3     vi val;
4
5     int search(int x){
6         // cnt >= x
7         auto it = lower_bound(all(val), x);
8         return val.end() - it; // #c83330
9         // cnt > x
10        //auto it = upper_bound(all(val), x);
11        //return val.end() - it; // #felbbd
12        // cnt == x
13        //auto l_it = lower_bound(all(val), x);
14        //auto u_it = upper_bound(all(val), x);
15        //return u_it - l_it; // #281439
16    }
17 };
18
19 Node operator+(Node a,Node b){
20     Node aux;
21     aux.val.resize(sz(a.val) + sz(b.val));
22     merge(all(a.val), all(b.val), aux.val.begin());
23 } // #086024

```

2.3 Segment Tree Recursive

```

1 //Hash: #f94ebe
2 struct Node{
3     ll val = 0;
4 };
5 Node operator+(Node a,Node b){return {a.val+b.val};}
6 class Segment_Tree{
7 public:
8     int n;
9     vector<Node> tree;
10
11     Segment_Tree(int x = 1e5+10){
12         n = x;
13         tree.resize(4*n+1);
14     } // #f71c07
15
16     void update(int v, int tl, int tr, int pos, int new_val){
17         if(tl == tr){
18             tree[v] = {new_val};
19             return;
20         }
21
22         int tm = tl+(tr-tl)/2;
23
24         if (pos<=tm)update(v<<1,tl,tm,pos,new_val);
25         else update((v<<1)+1,tm+1,tr,pos,new_val);
26
27         tree[v] = tree[v<<1] + tree[(v<<1)+1];
28     } // #b8d734

```

```

29
30 Node query(int v, int tl, int tr, int l, int r){
31     if (l > r)return {0};
32     if (l==tl && r==tr)return tree[v];
33
34     int tm = tl+(tr-tl)/2;
35     return query(v<<1,tl,tm,l,min(r,tm))+query((v<<1)+1,tm+1,tr,
36         max(l,tm+1),r);
37 } // #3499b3

```

2.4 Lazy

```

1 // Hash: c02121
2 struct Segment_Tree{
3
4     int n;
5     vector<Node> tree;
6     vector<Lazy> lazy; // #b7ee33
7
8     Segment_Tree(int x = 1e5+10){
9         n = x;
10        tree.resize(4*n);
11        lazy.resize(4*n);
12    } // #a86d70
13
14    void push_set(int v,int l,int r){
15        if(!lazy[v].has_set)return;
16        tree[v].val = lazy[v].val*(r-l+1);
17
18        if(l!=r){
19            lazy[v<<1] = lazy[v];
20            lazy[(v<<1)|1] = lazy[v];
21        }
22        lazy[v].has_set = 0;
23        lazy[v].val = 0;
24    } // #fd22cc
25
26    void push_sum(int v,int l,int r){
27        if(!lazy[v].has_sum)return;
28        tree[v].val = tree[v].val+lazy[v].val*(r-l+1);
29        if(l!=r){
30            lazy[v<<1].val = lazy[v<<1].val + lazy[v].val;
31            lazy[(v<<1)|1].val = lazy[(v<<1)|1].val + lazy[v].val;
32            if(!lazy[v<<1].has_set)lazy[v<<1].has_sum=1;
33            if(!lazy[(v<<1)|1].has_set)lazy[(v<<1)|1].has_sum=1;
34        }
35        lazy[v].has_sum = 0;
36        lazy[v].val= 0;
37    } // #a6dba9
38
39    void update(int pos,ll val,int v,int tl, int tr){
40        if(tl == tr)tree[v] = {val};
41        else{
42            int tm = tl+(tr-tl)/2;
43            if(pos <= tm)update(pos,val,v<<1,tl,tm);
44            else update(pos,val,(v<<1)+1,tm+1,tr);
45            tree[v] = tree[v<<1]+tree[(v<<1)+1];
46        }
47    } // #19c7ff
48
49    void update_set(int v,int tl,int tr,int l,int r,ll val){
50        push_set(v,tl,tr);
51        push_sum(v,tl,tr);
52        if(l>tr||r<tl)return;
53        if(tl>= l && tr <=r){
54            lazy[v].has_set=1;
55            lazy[v].has_sum=0;

```

```

56     lazy[v].val=val;
57     push_set(v,tl,tr);
58     return;
59 }
60 int tm = tl+(tr-tl)/2;
61 update_set(v<<1,tl,tm,l,r,val);
62 update_set((v<<1)|1,tm+1,tr,l,r,val);
63 tree[v] = tree[v<<1]+tree[(v<<1)|1];
64 } // #91ab68

65 void update_sum(int v,int tl,int tr,int l,int r,ll val){
66     push_set(v,tl,tr);
67     push_sum(v,tl,tr);
68     if(l>tr||r<tl)return;
69     if(tl>= l && tr <=r){
70         lazy[v].has_sum=1;
71         lazy[v].val += val;
72         push_sum(v,tl,tr);
73         return;
74     }
75     int tm = tl+(tr-tl)/2;
76     update_sum(v<<1,tl,tm,l,r,val);
77     update_sum((v<<1)|1,tm+1,tr,l,r,val);
78     tree[v] = tree[v<<1]+tree[(v<<1)|1];
79 } // #e2f028
80
81 Node query(int v ,int tl ,int tr ,int l ,int r){
82     push_set(v,tl,tr);
83     push_sum(v,tl,tr);
84     if(l>tr||r<tl)return {0};
85     if(tl>= l && tr <= r )
86         return tree[v];
87     int tm = tl+(tr-tl)/2;
88     return query(v<<1,tl,tm,l,r)+query((v<<1)|1,tm+1,tr,l,r);
89 } // #3cd2a5
90
91 };
92

```

2.5 Persistent Segment Tree

```

1 // Hash: #437d7b
2 struct Node{
3     ll val;
4     Node *l,*r;
5     Node():val(0),l(nullptr),r(nullptr){}
6 }; // #736465
7
8 ll get(Node* t){
9     return t?t->val:0;
10 } // #dea637
11
12 Node *update(Node *t,int tl,int tr,int pos,int v){
13     Node* aux = new Node();
14     if(tl == tr){
15         aux->val = v;
16         return aux;
17     }
18
19     int tm = tl+(tr-tl)/2;
20
21     Node* l = t?t->l:nullptr;
22     Node* r = t?t->r:nullptr;
23
24     if(pos <= tm){
25         aux->l = update(l,tl,tm,pos,v);
26         aux->r = r;
27     }else{
28         aux->l = l;

```

```

29         aux->r = update(r,tm+1,tr,pos,v);
30     }
31
32     aux->val = get(aux->l)+get(aux->r);
33     return aux;
34 } // #a8f7ef
35
36 ll query(Node *t,int tl,int tr,int l,int r){
37     if(!t || l>r)return 0;
38     if(l==tl && r == tr)return t->val;
39     int tm = tl+(tr-tl)/2;
40     return query(t->l,tl,tm,l,min(r,tm))+query(t->r,tm+1,tr,max(l,tm
41         +1),r);
42 } // #4596dc

```

2.6 Dynamic Segment Tree

```

1 struct Vertex {
2     int left, right;
3     int sum = 0;
4     Vertex *left_child = nullptr, *right_child = nullptr;
5     Vertex(int lb, int rb) {
6         left = lb;
7         right = rb;
8     }
9     void extend() {
10        if (!left_child && left + 1 < right) {
11            int t = (left + right) / 2;
12            left_child = new Vertex(left, t);
13            right_child = new Vertex(t, right);
14        }
15    }
16    void add(int k, int x) {
17        extend();
18        sum += x;
19        if (left_child) {
20            if (k < left_child->right)
21                left_child->add(k, x);
22            else
23                right_child->add(k, x);
24        }
25    }
26    int get_sum(int lq, int rq) {
27        if (lq <= left && right <= rq)
28            return sum;
29        if (max(left, lq) >= min(right, rq))
30            return 0;
31        extend();
32        return left_child->get_sum(lq, rq) + right_child->get_sum(lq, rq);
33    }
34 };

```

2.7 Sparse Table

```

1 //Hash: #a9e313
2 struct Node{
3     int val = 0;
4     Node operator+(Node b){return {__gcd(val,b.val)};}
5 }; // #f7df3b
6
7 class SparseTable{
8     public:
9     vector<vector<Node>> sparse;
10
11     SparseTable(int n=2e5+10){
12         sparse.assign(n , vector<Node> (log2(n)+1));
13     } // #031899
14

```

```

15 void build(const vi &a){
16     for(int i=0;i<sz(a);i++)
17         sparse[i][0] = {a[i]};
18
19     for(int j=1;(1<<j)<=sz(a);j++)
20         for(int i=0;i+(1<<j)-1 <sz(a);i++)
21             sparse[i][j]= sparse[i][j-1]+sparse[i+(1<<(j-1))][j-1];
22 } // #139c7b
23
24 int query(int l, int r){
25     int len = r-l+1;
26     int k = log2(len);
27     return (sparse[l][k]+sparse[r-(1<<k)+1][k]).val;
28 } // #db37a1
29 };

```

2.8 Treap

```

1 struct node {
2     int val, sz, prior, lazy, sum, mx, mn, repl;
3     bool repl_flag, rev;
4     node *l, *r, *par;
5     node() {
6         lazy = 0;
7         rev = 0;
8         sum = 0;
9         val = 0;
10        sz = 0;
11        mx = 0;
12        mn = 0;
13        repl = 0;
14        repl_flag = 0;
15        prior = 0;
16        l = NULL;
17        r = NULL;
18        par = NULL;
19    }
20    node(int _val) {
21        val = _val;
22        sum = _val;
23        mx = _val;
24        mn = _val;
25        repl = 0;
26        repl_flag = 0;
27        rev = 0;
28        lazy = 0;
29        sz = 1;
30        prior = rnd();
31        l = NULL;
32        r = NULL;
33        par = NULL;
34    }
35 };
36
37 struct treap {
38     typedef node* pnode;
39     pnode root;
40     map<int, pnode> position;//positions of all the values
41     //clearing the treap
42     void clear() {
43         root = NULL;
44         position.clear();
45     }
46
47     treap() {
48         clear();
49     }
50

```

```

51 int size(pnode t) {
52     return t ? t->sz : 0;
53 }
54 void update_size(pnode &t) {
55     if(t) t->sz = size(t->l) + size(t->r) + 1;
56 }
57
58 void update_parent(pnode &t) {
59     if(!t) return;
60     if(t->l) t->l->par = t;
61     if(t->r) t->r->par = t;
62 }
63 //add operation
64 void lazy_sum_upd(pnode &t) {
65     if( !t or !t->lazy ) return;
66     t->sum += t->lazy * size(t);
67     t->val += t->lazy;
68     t->mx += t->lazy;
69     t->mn += t->lazy;
70     if( t->l ) t->l->lazy += t->lazy;
71     if( t->r ) t->r->lazy += t->lazy;
72     t->lazy = 0;
73 }
74 //replace update
75 void lazy_repl_upd(pnode &t) {
76     if( !t or !t->repl_flag ) return;
77     t->val = t->mx = t->mn = t->repl;
78     t->sum = t->val * size(t);
79     if( t->l ) {
80         t->l->repl = t->repl;
81         t->l->repl_flag = true;
82     }
83     if( t->r ) {
84         t->r->repl = t->repl;
85         t->r->repl_flag = true;
86     }
87     t->repl_flag = false;
88     t->repl = 0;
89 }
90 //reverse update
91 void lazy_rev_upd(pnode &t) {
92     if( !t or !t->rev ) return;
93     t->rev = false;
94     swap(t->l, t->r);
95     if( t->l ) t->l->rev ^= true;
96     if( t->r ) t->r->rev ^= true;
97 }
98 //reset the value of current node assuming it now
99 //represents a single element of the array
100 void reset(pnode &t) {
101     if(!t) return;
102     t->sum = t->val;
103     t->mx = t->val;
104     t->mn = t->val;
105 }
106 //combine node l and r to form t by updating corresponding
107 //queries
108 void combine(pnode &t, pnode l, pnode r) {
109     if(!l) {
110         t = r;
111         return;
112     }
113     if(!r) {
114         t = l;
115         return;
116     }
117     //Beware!!!Here t can be equal to l or r anytime

```

```

117 //i.e. t and (l or r) is representing same node
118 //so operation is needed to be done carefully
119 //e.g. if t and r are same then after t->sum=l->sum+r->sum
120 //operation,
121 //r->sum will be the same as t->sum
122 //so BE CAREFUL
123 t->sum = l->sum + r->sum;
124 t->mx = max(l->mx, r->mx);
125 t->mn = min(l->mn, r->mn);
126 }
127 //perform all operations
128 void operation(pnode &t) {
129     if( !t ) return;
130     reset(t);
131     lazy_rev_upd(t->l);
132     lazy_rev_upd(t->r);
133     lazy_repl_upd(t->l);
134     lazy_repl_upd(t->r);
135     lazy_sum_upd(t->l);
136     lazy_sum_upd(t->r);
137     combine(t, t->l, t);
138     combine(t, t, t->r);
139 }
140 //split node t in l and r by key k
141 //so first k+1 elements(0,1,2,...k) of the array from node t
142 //will be split in left node and rest will be in right node
143 void split(pnode t, pnode &l, pnode &r, int k, int add = 0) {
144     if(t == null) {
145         l = null;
146         r = null;
147         return;
148     }
149     lazy_rev_upd(t);
150     lazy_repl_upd(t);
151     lazy_sum_upd(t);
152     int idx = add + size(t->l);
153     if(t->l) t->l->par = null;
154     if(t->r) t->r->par = null;
155     if(idx <= k)
156         split(t->r, t->r, r, k, idx + 1), l = t;
157     else
158         split(t->l, l, t->l, k, add), r = t;
159
160     update_parent(t);
161     update_size(t);
162     operation(t);
163 }
164 //merge node l with r in t
165 void merge(pnode &t, pnode l, pnode r) {
166     lazy_rev_upd(l);
167     lazy_rev_upd(r);
168     lazy_repl_upd(l);
169     lazy_repl_upd(r);
170     lazy_sum_upd(l);
171     lazy_sum_upd(r);
172     if(!l) {
173         t = r;
174         return;
175     }
176     if(!r) {
177         t = l;
178         return;
179     }
180
181     if(l->prior > r->prior)
182         merge(l->r, l->r, r), t = l;
183     else

```

```

183     merge(r->l, l, r->l), t = r;
184
185     update_parent(t);
186     update_size(t);
187     operation(t);
188 }
189 //insert val in position a[pos]
190 //so all previous values from pos to last will be right shifted
191 void insert(int pos, int val) {
192     if(root == NULL) {
193         pnode to_add = new node(val);
194         root = to_add;
195         position[val] = root;
196         return;
197     }
198
199     pnode l, r, mid;
200     mid = new node(val);
201     position[val] = mid;
202
203     split(root, l, r, pos - 1);
204     merge(l, l, mid);
205     merge(root, l, r);
206
207     /*pnode x = new Node(val),merge(root,root,x);*/
208 }
209 //erase from qL to qR indexes
210 //so all previous indexes from qR+1 to last will be left shifted
211 //qR-qL+1 times
212 void erase(int qL, int qR) {
213     pnode l, r, mid;
214
215     split(root, l, r, qL - 1);
216     split(r, mid, r, qR - qL);
217     merge(root, l, r);
218 }
219 //returns answer for corresponding types of query
220 int query(int qL, int qR) {
221     pnode l, r, mid;
222
223     split(root, l, r, qL - 1);
224     split(r, mid, r, qR - qL);
225
226     int answer = mid->sum;
227     merge(r, mid, r);
228     merge(root, l, r);
229
230     return answer;
231 }
232 //add val in all the values from a[qL] to a[qR] positions
233 void update(int qL, int qR, int val) {
234     pnode l, r, mid;
235
236     split(root, l, r, qL - 1);
237     split(r, mid, r, qR - qL);
238     lazy_repl_upd(mid);
239     mid->lazy += val;
240
241     merge(r, mid, r);
242     merge(root, l, r);
243 }
244 //reverse all the values from qL to qR
245 void reverse(int qL, int qR) {
246     pnode l, r, mid;
247
248     split(root, l, r, qL - 1);
249     split(r, mid, r, qR - qL);

```

```

249     if(mid)mid->rev ^= 1;
250     merge(r, mid, r);
251     merge(root, l, r);
252 }
253 //replace all the values from a[qL] to a[qR] by v
254 void replace(int qL, int qR, int v) {
255     pnode l, r, mid;
256
257     split(root, l, r, qL - 1);
258     split(r, mid, r, qR - qL);
259     lazy_sum_upd(mid);
260     mid->repl_flag = 1;
261     mid->repl = v;
262     merge(r, mid, r);
263     merge(root, l, r);
264 }
265 //it will cyclic right shift the array k times
266 //so for k=1, a[qL]=a[qR] and all positions from qL+1 to qR will
267 //have values from previous a[qL] to a[qR-1]
268 //if you make left_shift=1 then it will to the opposite
269 void cyclic_shift(int qL, int qR, int k, bool left_shift = 0) {
270     if(qL == qR) return;
271     k %= (qR - qL + 1);
272
273     pnode l, r, mid, fh, sh;
274     split(root, l, r, qL - 1);
275     split(r, mid, r, qR - qL);
276
277     if(left_shift == 0) split(mid, fh, sh, (qR - qL + 1) - k - 1);
278     else split(mid, fh, sh, k - 1);
279     merge(mid, sh, fh);
280
281     merge(r, mid, r);
282     merge(root, l, r);
283 }
284 bool exist;
285 //returns index of node curr
286 int get_pos(pnode curr, pnode son = nullptr) {
287     if(exist == 0) return 0;
288     if(curr == NULL) {
289         exist = 0;
290         return 0;
291     }
292     if(!son) {
293         if(curr == root) return size(curr->l);
294         else return size(curr->l) + get_pos(curr->par, curr);
295     }
296
297     if(curr == root) {
298         if(son == curr->l) return 0;
299         else return size(curr->l) + 1;
300     }
301
302     if(curr->l == son) return get_pos(curr->par, curr);
303     else return get_pos(curr->par, curr) + size(curr->l) + 1;
304 }
305 //returns index of the value
306 //if the value has multiple positions then it will
307 //return the last index where it was added last time
308 //returns -1 if it doesn't exist in the array
309 int get_pos(int value) {
310     if(position.find(value) == position.end()) return -1;
311     exist = 1;
312     int x = get_pos(position[value]);
313     if(exist == 0) return -1;
314     else return x;

```

```

316 }
317 //returns value of index pos
318 int get_val(int pos) {
319     return query(pos, pos);
320 }
321 //returns size of the treap
322 int size() {
323     return size(root);
324 }
325 //inorder traversal to get indexes chronologically
326 void inorder(pnode cur) {
327     if(cur == NULL) return;
328     operation(cur);
329     inorder(cur->l);
330     cout << cur->val << ' ';
331     inorder(cur->r);
332 }
333
334 bool find(int val) {
335     if(get_pos(val) == -1) return 0;
336     else return 1;
337 }
338 };
339
340 treap t;
341 //Beware!!!here treap is 0-indexed
342
343 int main() {
344     int i, j, k, n, m, l, r, q;
345     for(i = 0; i < 10; i++) t.insert(i, i * 10);
346     t.cyclic_shift(4, 5, 1);
347     t.update(2, 5, 1);
348     t.replace(2, 5, 100);
349     t.reverse(2, 9);
350     t.replace(2, 5, 200);
351     cout << t.query(0, 7) << nl;
352     t.cyclic_shift(2, 3, 2, 1);
353     cout << t.get_pos(20) << nl;
354     t.erase(2, 2);
355     cout << t.find(30) << nl;
356     t.print_array();
357     return 0;
358 }

```

2.9 BIT

```

1 // Hash: b34e73
2 struct BIT{
3     vll bit;
4     int n;
5     BIT(int x){n = x+1;bit.assign(n,0);} // #69335b
6
7     ll get(int x){
8         ll ans =0;
9         for(;x>=x&-x)ans+=bit[x];
10        return ans;
11    } // #781a0f
12
13    void update(int x, ll val){
14        for(;x<n;x+=x&-x)
15            bit[x] += val;
16    } // #4e7832
17 };

```

2.10 DSU

```

1 // Hash: 60e0d3
2 class DSU{

```

```

3 private:
4     vi parent,rank,size;
5     int c;
6 public:
7     int mx = 0; // #9e8891
8     DSU(int n):parent(n+1),rank(n+1,0),size(n+1,1),c(n){
9         iota(all(parent),0);
10    } // #9dff24
11
12    int find(int u){return parent[u] == u?u:parent[u] = find(parent[
13        u]);} // #a9e220
14
15    bool same(int u,int v){return find(u)==find(v);} // #df0aa8
16
17    int get_size(int u){return size[find(u)];} // #f7b590
18
19    int count(){return c;} // #466020
20
21    void merge(int u,int v){
22        u = find(u);
23        v = find(v);
24        if(u!=v){
25            c--;
26            if(rank[u] > rank[v])swap(u,v);
27            parent[u] = v;
28            size[v] += size[u];
29            if(rank[u] == rank[v])rank[v]++;
30            mx = max(mx,size[v]);
31        } // #ee7199
32    };

```

2.11 DSU with Rollback

```

1 class DSU{
2 private:
3     vi parent,rank,size;
4     int c;
5     struct State{
6         int u,v;
7         int rank_v,size_v,mx;
8     };
9     vector<State> history;
10
11 public:
12     int mx = 0;
13     DSU(int n):parent(n+1),rank(n+1,0),size(n+1,1),c(n){
14         iota(all(parent),0);
15     }
16
17     int find(int u){
18         while(u!=parent[u])u = parent[u];
19         return u;
20     }
21
22     bool same(int u,int v){return find(u)==find(v);} // #df0aa8
23
24     int get_size(int u){return size[find(u)];} // #f7b590
25
26     int count(){return c;} // #466020
27
28     int get_time(){return history.size();}
29
30     void rollback(int t){
31         while(history.size()>t){
32             State st = history.back();
33             history.pop_back();
34             int u = st.u;

```

```

35     int v = st.v;
36     parent[u] = u;
37     rank[v] = st.rank_v;
38     size[v] = st.size_v;
39     mx = st.mx;
40     c++;
41 }
42 }
43
44 void rollback(){
45     if(history.empty())return;
46     State st = history.back();
47     history.pop_back();
48     int u = st.u;
49     int v = st.v;
50     parent[u] = u;
51     rank[v] = st.rank_v;
52     size[v] = st.size_v;
53     mx = st.mx;
54     c++;
55 }
56
57 void merge(int u,int v){
58     u = find(u);
59     v = find(v);
60     if(u!=v){
61         c--;
62         if(rank[u] > rank[v])swap(u,v);
63         history.pb({u,v,rank[v],size[v],mx});
64         parent[u] = v;
65         size[v] += size[u];
66         if(rank[u] == rank[v])rank[v]++;
67         mx = max(mx,size[v]);
68     }
69 }
70 };

```

2.12 Dynamic Connectivity

```

1 struct Dynaco{
2     int n;
3     vector<vector<pii>> st;
4     DSU dsu;//with rollbacks
5
6     Dynaco(int T,int v):n(T),dsu(v){st.resize(4*n+1);}
7
8     void add(int v, int tl, int tr, int l, int r, pii e){
9         if(l>r)return;
10
11         if(l==tl&&r==tr){
12             st[v].push_back(e);
13             return;
14         }
15         int tm = tl+(tr-tl)/2;
16         add(v<<1,tl,tm,l,min(r,tm),e);
17         add((v<<1)+1,tm+1,tr,max(l,tm+1),r,e);
18     }
19
20
21     void dfs(int v, int tl, int tr, vi& ans){
22         int merges = 0;
23
24         for(auto [x,y]:st[v])
25             if(dsu.merge(x,y))merges++;
26
27         if(tl==tr){
28             ans[tl] += dsu.count();
29         }else{

```

```

30         int tm = tl+(tr-tl)/2;
31         dfs(v<<1,tl,tm,ans);
32         dfs((v<<1)+1,tm+1,tr,ans);
33     }
34
35     while(merges-->0)dsu.rollback();
36 }
37 };

```

2.13 Trie

```

1 struct Node{
2     int v;
3     int cnt = 0;
4     Node* nxt[2];
5     Node(int __v){
6         v = __v;
7         cnt = 0;
8         nxt[0] = nxt[1] = NULL;
9     }
10 };
11
12 struct Trie{
13     typedef Node* pnode;
14     pnode root;
15     const int B = 30;
16
17     void insert(int val){
18         pnode cur = root;
19         for(int i=B;i>0;i--){
20             bool b = val>>i&1;
21             if(cur->nxt[b]==NULL)cur->nxt[b] = new Node(b);
22             cur = cur->nxt[b];
23             cur->cnt++;
24         }
25     }
26
27     pnode erase(pnode cur, int val, int i) {
28         if (cur == NULL)return NULL;
29         cur->cnt--;
30         if (i < 0) {
31             if (cur->cnt == 0) {
32                 delete cur;
33                 return NULL;
34             }
35             return cur;
36         }
37
38         bool b = (val >> i) & 1;
39         cur->nxt[b] = erase(cur->nxt[b], val, i - 1);
40
41         if (cur->cnt == 0) {
42             delete cur;
43             return NULL;
44         }
45
46         return cur;
47     }
48
49     int query(int val){
50         pnode cur = root;
51         int ans = 0;
52         for(int i = B;i>0;i--){
53             bool b = val>>i&1;
54             if(cur->nxt[b])cur = cur->nxt[b],ans<=1,ans++;
55             else cur = cur->nxt[b],ans<=1;
56         }
57         return ans;

```

```

58     }
59 };

```

2.14 Mo's Algorithm

```

1 void remove(idx); // TODO: remove value at idx from data
   structure
2 void add(idx); // TODO: add value at idx from data structure
3 int get_answer(); // TODO: extract the current answer of the data
   structure
4 // #86ec16
5 int block_size;
6
7 struct Query{
8     int l, r, idx;
9     bool operator<(Query other) const
10 {
11     return make_pair(l / block_size, r) <
12            make_pair(other.l / block_size, other.r);
13 }
14 }; // #8e89a9
15
16 //in main
17
18 sort(all(queries));
19
20 // TODO: initialize data structure
21
22 int cur_l = 0;
23 int cur_r = -1;
24 // invariant: data structure will always reflect the range [cur_l,
   cur_r]
25 for(Query q:queries){
26     while(cur_l > q.l)
27         add(--cur_l);
28
29     while(cur_r < q.r)
30         add(++cur_r);
31
32     while(cur_l < q.l)
33         remove(cur_l++);
34
35     while(cur_r > q.r)
36         remove(cur_r--);
37
38     answers[q.idx] = get_answer();
39 } // #50d13b

```

3 Graphs

3.1 2-SAT

```

1 class SAT{
2 private:
3     int n;
4     vector<vi> adj;
5     vector<vi> adj_t;
6     vector<vi> adj_cond;
7     vi order;
8     vector<bool> vis;
9     vi who;
10    vector<bool> ans;
11    int comp;
12 public:
13
14    SAT(int m=0){
15        n = (m<<1);
16        adj.resize(n);

```



```

17 adj_t.resize(n);
18 vis.assign(n,0);
19 who.assign(n,-1);
20 }
21
22 void dfs1(int u) {
23     vis[u] = true;
24     for (int v:adj[u])
25         if (!vis[v])dfs1(v);
26     order.pb(u);
27 }
28
29 void dfs2(int u) {
30     vis[u] = 1;
31     for (int v : adj_t[u])
32         if (!vis[v])dfs2(v);
33     who[u] = comp;
34 }
35
36 //~a->b ~b->a
37 void addOR(int a, bool af, int b, bool bf) {
38     a += a + (af ^ 1);
39     b += b + (bf ^ 1);
40     adj[a ^ 1].push_back(b);
41     adj[b ^ 1].push_back(a);
42     adj_t[b].push_back(a ^ 1);
43     adj_t[a].push_back(b ^ 1);
44 }
45
46 //( a OR b ) ^ (~a or ~b)
47 void addXOR(int a, bool af, int b, bool bf) {
48     addOR(a, af, b, bf);
49     addOR(a, !af, b, !bf);
50 }
51 //(a -> b)
52 void _add(int a,bool af,int b,bool bf) {
53     a += a + (af ^ 1);
54     b += b + (bf ^ 1);
55     adj[a].push_back(b);
56     adj_t[b].push_back(a);
57 }
58
59 //(a->b)^(b->a)
60 void add(int a,bool af,int b,bool bf) {
61     _add(a, af, b, bf);
62     _add(b, !bf, a, !af);
63 }
64
65 bool solve_SAT(){
66     for(int i=0;i<n;i++){
67         if(!vis[i])dfs1(i);
68
69         comp = -1;
70         vis.assign(n,0);
71         for(int i=sz(order)-1;i>=0;i--){
72             if(!vis[order[i]]){
73                 comp++;
74                 dfs2(order[i]);
75             }
76             ans.assign(n/2,0);
77             for(int i=0;i<n;i+=2){
78                 if(who[i] == who[i+1])return 0;
79                 ans[i/2] = (who[i] > who[i+1]);
80             }
81             return 1;
82         }
83     }

```

```

84     bool operator[](int i){
85         return ans[i];
86     }
87
88 };

```

3.2 Block Cut Tree

```

1 vector<vi> adj(n+1),bcc(n+1);
2 vi disc(n+1),low(n+1),who(n+1);
3 stack<int> st;
4 int Time = 0,sz = 0,x;
5
6 auto dfs = [&](auto &self,int u,int p)->void{
7     disc[u] = low[u] = ++Time;
8     st.push(u);
9     for(int v:adj[u]){
10         if(!disc[v]){
11             self(self,v,u);
12             low[u] = min(low[u],low[v]);
13             if(low[v] >= disc[u]){
14                 sz++;
15                 do{
16                     x = st.top();
17                     st.pop();
18                     bcc[x].pb(sz);
19                 }while(x^v);
20                 bcc[u].pb(sz);
21             }
22         }else if(v!=p)low[u] = min(low[u],disc[v]);
23     }
24 };
25
26 for(int i=1;i<=n;i++){
27     if(!disc[i])dfs(dfs,i,0);
28 }
29 int c = sz+n+2;
30 vector<vi> tree(c);
31 vector<bool> art(c,0);
32 for(int i=1;i<=n;i++){
33     if(sz(bcc[i])>1){
34         who[i] = ++sz;
35         art[who[i]] = 1;
36         for(int v:bcc[i]){
37             tree[who[i]].pb(v);
38             tree[v].pb(who[i]);
39         }
40     }else if(sz(bcc[i])==1)who[i] = bcc[i][0];
41 }

```

3.3 Bellman-Ford Cycle

```

1 vector<int> d(n, INF);
2 d[v] = 0;
3 vector<int> p(n, -1);
4 int x;
5 for (int i = 0; i < n; ++i) {
6     x = -1;
7     for (Edge e : edges)
8         if (d[e.a] < INF)
9             if (d[e.b] > d[e.a] + e.cost) {
10                 d[e.b] = max(-INF, d[e.a] + e.cost);
11                 p[e.b] = e.a;
12                 x = e.b;
13             }
14 }
15 if (x == -1)cout << "No negative cycle from " << v;
16 else{

```

```

17 int y = x;
18 for (int i = 0; i < n; ++i)y = p[y];
19 vector<int> path;
20 for (int cur = y;; cur = p[cur]) {
21     path.push_back(cur);
22     if (cur == y && path.size() > 1)break;
23 }
24 reverse(path.begin(), path.end());
25 }

```

3.4 Bellman-Ford Path

```

1 vector<int> d(n, INF);
2 d[v] = 0;
3 vector<int> p(n, -1);
4
5 for (;) {
6     bool any = false;
7     for (Edge e : edges)
8         if (d[e.a] < INF)
9             if (d[e.b] > d[e.a] + e.cost) {
10                 d[e.b] = d[e.a] + e.cost;
11                 p[e.b] = e.a;
12                 any = true;
13             }
14     if (!any)
15         break;
16 }
17
18 if (d[t] == INF)
19     cout << "No path from " << v << " to " << t << ".";
20 else {
21     vector<int> path;
22     for (int cur = t; cur != -1; cur = p[cur])path.push_back(cur);
23     reverse(path.begin(), path.end());
24 }

```

3.5 Bipartite Check

```

1 vector<vi> color;
2 bool ok = 1;
3 queue<int> q;
4 for(int i=0;i<n;i++){
5     if(a[i]!=0)continue;
6     q.push(i);
7     color[i] = 1;
8     while (!q.empty()) {
9         int v = q.front();
10        q.pop();
11        for(int u:adj[v]){
12            if(a[u]==0){
13                a[u] = a[v]^3;
14                q.push(u);
15            }else{
16                valid &= a[u]!=a[v];
17            }
18        }
19    }
20 }

```

3.6 Bridges

```

1
2 vector<pii> bridge;
3 vi disc(MAX),low(MAX);
4 vector<vi> adj(MAX);
5 int Time = 0;
6
7 void dfs(int u,int p){

```

```

8  disc[u] = low[u] = ++Time;
9  for(int v:adj[u]){
10     if(v==p)continue;
11     if(!disc[v]){
12         dfs(v,u);
13         // <= for articulation points
14         if(disc[u]<low[v])bridge.emplace_back(u,v);
15         low[u] = min(low[u],low[v]);
16     }else{
17         low[u] = min(low[u],disc[v]);
18     }
19 }
20 }
21 }

```

3.7 Bridge Tree

```

1  //Bridges already found
2  vi who(n+1,-1);
3  int c = 1;
4
5  auto dfs2 = [&](auto &self,int u)->void{
6      who[u] = c;
7      for(auto [v,i]:adj[u]){
8          if(who[v]!=-1||bridge[i])continue;
9          self(self,v);
10     }
11 };
12
13
14 for(int i=1;i<=n;i++){
15     if(who[i]==-1)dfs2(dfs2,i),c++;
16 }
17
18 vector<vi> tree(c);
19
20 for(int i=0;i<m;i++){
21     if(bridge[i]){
22         tree[who[edges[i].u]].pb(who[edges[i].v]);
23         tree[who[edges[i].v]].pb(who[edges[i].u]);
24     }
25 }

```

3.8 Dijkstra

```

1  vll dist(n,INF64);
2  vector<bool> vis(n,0);
3  priority_queue<pll,vector<pll>,greater<pll>> pq;
4  pq.emplace(0,0);
5  dist[0] = 0;
6  vll cnt(n,0); //number of shortest paths
7  while(!pq.empty()){
8      auto [d1,u] = pq.top();
9      pq.pop();
10     if(vis[u])continue;
11     vis[u] = 1;
12     for(auto [v,c]:adj[u]){
13         if(dist[u]+c < dist[v]){
14             dist[v] = dist[u]+c;
15             pq.emplace(dist[v],v);
16             cnt[v] = cnt[u];
17         }else if(dist[u]+c == dist[v])(cnt[v]+=cnt[u])%=MOD;
18     }
19 }

```

3.9 Dinic

```

1  struct flowEdge{
2  int u,v;

```

```

3  ll cap,flow = 0;
4  flowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap) {};
5  };
6
7  struct Dinic{
8  vector<flowEdge> edges;
9  vector<vi> adj;
10 int n,s,t;
11 int id = 0;
12 vi level, next;
13
14 queue<int> q;
15
16 Dinic(int n, int s, int t) : n(n), s(s), t(t) {
17     adj.resize(n);
18     level.resize(n);
19     next.resize(n);
20     fill(all(level),-1);
21     level[s] = 0;
22     q.push(s);
23 }
24
25 void add(int u, int v, ll cap){
26     edges.emplace_back(u,v,cap);
27     edges.emplace_back(v,u,0);
28     adj[u].pb(id++);
29     adj[v].pb(id++);
30 }
31
32 bool bfs(){
33     while (!q.empty()){
34         int curr = q.front();
35         q.pop();
36         for (auto e : adj[curr]){
37             if (edges[e].cap - edges[e].flow < 1) continue;
38             if (level[edges[e].v] != -1) continue;
39             level[edges[e].v] = level[edges[e].u] + 1;
40             q.push(edges[e].v);
41         }
42     }
43     return level[t] != -1;
44 }
45
46 ll dfs(int u, ll flow){
47     if (flow == 0) return 0;
48     if (u == t) return flow;
49     for (auto& cid = next[u]; cid < adj[u].size(); cid++){
50         int e = adj[u][cid];
51         int v = edges[e].v;
52
53         if (level[edges[e].u] + 1 != level[v] || edges[e].cap -
edges[e].flow < 1) continue;
54         ll f = dfs(v,min(flow,edges[e].cap - edges[e].flow));
55         if (f == 0) continue;
56         edges[e].flow += f;
57         edges[e ^ 1].flow -= f;
58         return f;
59     }
60     return 0;
61 }
62
63 ll maxFlow(){
64     ll flow = 0;
65     while (bfs()){
66         fill(all(next),0);
67         for (ll f = dfs(s,INF64); f != 0ll; f = dfs(s,INF64)) flow
+= f;

```

```

68         fill(all(level),-1);
69         level[s] = 0;
70         q.push(s);
71     }
72     return flow;
73 }
74
75
76 vector<pii> minCut(){
77     vector<pii> ans;
78     fill(all(level),-1);
79     level[s] = 0;
80     q.push(s);
81     while (!q.empty()){
82         int curr = q.front();
83         q.pop();
84         for (int id = 0; id < adj[curr].size(); id++){
85             int e = adj[curr][id];
86             if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow
> 0){
87                 q.push(edges[e].v);
88                 level[edges[e].v] = level[edges[e].u] + 1;
89             }
90         }
91     }
92
93     for (int i = 0; i < level.size(); i++){
94         if (level[i] != - 1){
95             for (int id = 0; id < adj[i].size(); id++){
96                 int e = adj[i][id];
97                 if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow
== 0)
98                     ans.emplace_back(edges[e].u,edges[e].v);
99             }
100         }
101     }
102     return ans;
103 }
104
105 vector<pii> MaximumMatching(){
106     vector<pii> ans;
107     fill(all(level),-1);
108     level[s] = 0;
109     q.push(s);
110     while (!q.empty()){
111         int curr = q.front();
112         q.pop();
113         for (int id = 0; id < adj[curr].size(); id++){
114             int e = adj[curr][id];
115             if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow ==
0 && edges[e].flow != 0ll){
116                 q.push(edges[e].v);
117                 level[edges[e].v] = level[edges[e].u] + 1;
118             }
119         }
120     }
121     for (int i = 0; i < level.size(); i++){
122         if (level[i] != - 1)
123             for (int id = 0; id < adj[i].size(); id++){
124                 int e = adj[i][id];
125                 if (edges[e].u != s && edges[e].v != t && edges[e].cap -
edges[e].flow == 0 && edges[e].flow != 0ll)
126                     ans.emplace_back(edges[e].u,edges[e].v);
127             }
128     }
129     return ans;
130 }

```



```
131
132 };
```

3.10 Euler Path Directed

```
1 struct EulerSolver{
2 private:
3 vector<vector<pii>> adj;
4 int n,edges;
5 vi in,out;
6 vi ans,edge_ans;
7 int root;
8 vi done;
9 void dfs(int u){
10     while(done[u] < sz(adj[u])){
11         auto [v,id] = adj[u][done[u]++];
12         dfs(v);
13         edge_ans.pb(id);
14     }
15     ans.pb(u);
16 }
17 public:
18 EulerSolver(vector<vector<pii>> &a,int t):adj(a){
19     n = t;
20     edges = 0;
21     in.assign(n,0);
22     out.assign(n,0);
23     done.assign(n,0);
24     root = -1;
25     for(int u=0;u<n;u++){
26         for(auto v:adj[u]){
27             edges++;
28             in[v.F]++;
29             out[u]++;
30         }
31     }
32 }
33
34 bool possible(){
35     int cnt1 = 0,cnt2 = 0;
36     bool ok = 1;
37     for(int i=0;i<n;i++){
38         if (in[i] - out[i] == 1) cnt1++;
39         if (out[i] - in[i] == 1) cnt2++;, root = i;
40         if (abs(in[i] - out[i]) > 1) ok = 0;
41     }
42     if(cnt1 > 1 || cnt2 > 1)ok = 0;
43     if(!ok)return 0;
44
45     if(root == -1)
46         for(int i=0;i<n;i++)
47             if(out[i])root = i;
48
49     if(root == -1)return 1;
50     dfs(root);
51
52     if(sz(ans) != edges+1)return 0;
53     reverse(all(ans));
54     reverse(all(edge_ans));
55     return 1;
56 }
57
58 int size(){
59     return ans.size();
60 }
61
62 int operator()(int i,bool x){
63     if(x)return ans[i];
```

```
64     else return edge_ans[i];
65 }
66 };
```

3.11 Euler Path Undirected

```
1 struct EulerSolver{
2 private:
3 vector<vector<pii>> adj;
4 int n,edges;
5 vi ans,edge_ans;
6 vi deg;
7 int root;
8 vi done;
9 vector<bool> vis;
10
11 void dfs(int u){
12     while(done[u] < sz(adj[u])){
13         auto [v,id] = adj[u][done[u]++];
14         if(vis[id])continue;
15         vis[id]=1;
16         dfs(v);
17         edge_ans.pb(id);
18     }
19     ans.pb(u);
20 }
21 public:
22 EulerSolver(vector<vector<pii>> &a,int t):adj(a),n(t){
23     edges = 0;
24     done.assign(n,0);
25     deg.assign(n,0);
26     root = -1;
27     for(int u=0;u<n;u++){
28         for(auto v:adj[u]){
29             edges++;
30             deg[v.F]++;
31             deg[u]++;
32         }
33         vis.assign(edges/2,0);
34     }
35
36     bool possible(){
37         int odd = 0;
38         for(int i=0;i<n;i++){
39             if((deg[i]/2)&1){
40                 odd++;
41                 root = i;
42             }
43
44             if(odd>2)return 0;
45
46             if(root == -1)
47                 for(int i=0;i<n;i++)
48                     if(deg[i]){root = i;break;}
49
50             if(root == -1)return 1;
51             dfs(root);
52
53             if(sz(ans) != edges/2+1)return 0;
54             reverse(all(ans));
55             reverse(all(edge_ans));
56             return 1;
57         }
58     }
59
60     int size(){
61         return ans.size();
62     }
```

```
63
64     int operator()(int i,bool x){
65         if(x)return ans[i];
66         else return edge_ans[i];
67     }
68 };
```

3.12 Flood Fill

```
1 int dx[] = {1,1,0,-1,-1,-1,0,1};
2 int dy[] = {0,1,1,1,0,-1,-1,-1};
3
4 int floodfill(int x,int y,char color1,char color2){
5     if(grid[x][y] == "outside")return 0;
6     if(grid[x][y] != color1)return 0;
7     int ans = 1;
8     grid[x][y] = color2;
9     for(int d=0;d<8;d++)ans += floodfill(x+dx[d],y+dy[d],color1,
10         color2);
11     return ans;
12 }
```

3.13 Floyd-Warshall

```
1 for (int i = 0; i < n; ++i)d[i][i] = 0;
2
3 for (int k = 0; k < n; ++k)
4     for (int i = 0; i < n; ++i)
5         for (int j = 0; j < n; ++j)
6             if (d[i][k] < INF && d[k][j] < INF)
7                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

3.14 Kruskal

```
1 struct Edge{
2     int u,v,w;
3     bool operator < (Edge b){return w<b.w;}
4 };
5
6 //inside main
7
8 vector<Edge> edges;
9 sort(all(edges));
10
11 DSU dsu(vertices);
12
13 for(int i=0;i<E;i++){
14     Edge front = edges[i];
15     if(!dsu.same(front.u,front.v)){
16         cost+= front.w;
17         dsu.merge(front.u,front.v);
18     }
19 }
```

3.15 Maximum Clique

```
1 typedef vector<bitset<200>> vb;
2 struct Maxclique{
3     double limit=0.025, pk=0;
4     struct Vertex { int i, d=0; };
5     typedef vector<Vertex> vv;
6     vb e;
7     vv V;
8     vector<vi> C;
9     vi qmax, q, S, old;
10     void init(vv& r) {
11         for (auto& v : r) v.d = 0;
12         for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
13         sort(all(r), [](auto a, auto b) { return a.d > b.d; });
```

```

14     int mxD = r[0].d;
15     for(int i=0;i<sz(r);i++) r[i].d = min(i, mxD) + 1;
16 }
17 void expand(vv& R, int lev = 1) {
18     S[lev] += S[lev - 1] - old[lev];
19     old[lev] = S[lev - 1];
20     while (sz(R)) {
21         if (sz(q) + R.back().d <= sz(qmax)) return;
22         q.push_back(R.back().i);
23         vv T;
24         for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
25         if (sz(T)) {
26             if (S[lev]++ / ++pk < limit) init(T);
27             int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
28             C[1].clear(), C[2].clear();
29             for (auto v : T) {
30                 int k = 1;
31                 auto f = [&](int i) { return e[v.i][i]; };
32                 while (any_of(all(C[k]), f)) k++;
33                 if (k > mxk) mxk = k, C[mxk + 1].clear();
34                 if (k < mnk) T[j++].i = v.i;
35                 C[k].push_back(v.i);
36             }
37             if (j > 0) T[j - 1].d = 0;
38             for(int k=mnk;k<mxk + 1;k++) for (int i : C[k])
39                 T[j].i = i, T[j++].d = k;
40             expand(T, lev + 1);
41         } else if (sz(q) > sz(qmax)) qmax = q;
42         q.pop_back(), R.pop_back();
43     }
44 }
45 vi maxClique() { init(V), expand(V); return qmax; }
46 Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
47     for(int i=0;i<sz(e);i++)V.push_back({i});
48 }
49 };

```

3.16 MCMF

```

1 struct MCMF{
2     struct edge{
3         int u, v;
4         ll cap, cost;
5         int id;
6         edge(int _u, int _v, ll _cap, ll _cost, int _id) {
7             u = _u;
8             v = _v;
9             cap = _cap;
10            cost = _cost;
11            id = _id;
12        }
13    };
14
15    int n, s, t, mxid;
16    ll flow, cost;
17    vector<vi> g;
18    vector<edge> e;
19    vll d, potential, flow_through;
20    vi par;
21    bool neg;
22
23    MCMF(){}
24    /* indexed
25    MCMF(int _n){
26        n = _n + 10;
27        g.assign(n, vector<int> ());
28        neg = false;
29        mxid = 0;

```

```

30    }
31
32    void add_edge(int u, int v, ll cap, ll cost, int id = -1, bool
33        directed = true) {
34        if(cost < 0) neg = true;
35        g[u].push_back(e.size());
36        e.push_back(edge(u, v, cap, cost, id));
37        g[v].push_back(e.size());
38        e.push_back(edge(v, u, 0, -cost, -1));
39        mxid = max(mxid, id);
40        if(!directed) add_edge(v, u, cap, cost, -1, true);
41    }
42
43    bool dijkstra(){
44        par.assign(n, -1);
45        d.assign(n, INF64);
46        priority_queue<pll,vector<pll>,greater<pll>> q;
47        d[s] = 0;
48        q.push({0, s});
49
50        while (!q.empty()){
51            auto [nw,u] = q.top();
52            q.pop();
53            if(nw != d[u])continue;
54            for (int i = 0; i < sz(g[u]); i++) {
55                int id = g[u][i];
56                int v = e[id].v;
57                ll cap = e[id].cap;
58                ll w = e[id].cost + potential[u] - potential[v];
59                if (d[u]+w < d[v] && cap > 0) {
60                    d[v] = d[u] + w;
61                    par[v] = id;
62                    q.push({d[v], v});
63                }
64            }
65
66            for (int i = 0; i < n; i++){
67                if (d[i] < INF64)
68                    d[i] += (potential[i] - potential[s]);
69            }
70
71            for (int i = 0; i < n; i++){
72                if (d[i] < INF64) potential[i] = d[i];
73            }
74            return d[t] != INF64; // for max flow min cost
75            // return d[t] <= 0; // for min cost flow
76        }
77
78        ll send_flow(int v, ll cur){
79            if(par[v] == -1) return cur;
80            int id = par[v];
81            int u = e[id].u;
82            ll w = e[id].cost;
83            ll f = send_flow(u, min(cur, e[id].cap));
84            cost += f * w;
85            e[id].cap -= f;
86            e[id ^ 1].cap += f;
87            return f;
88        }
89
90        pll solve(int _s, int _t, ll goal = INF64) {
91            s = _s;
92            t = _t;
93            flow = 0, cost = 0;
94            potential.assign(n, 0);
95            if(neg){

```

```

96            d.assign(n, INF);
97            d[s] = 0;
98            bool relax = true;
99            for (int i = 0; i < n && relax; i++) {
100                relax = false;
101                for (int u = 0; u < n; u++) {
102                    for (int k = 0; k < (int)g[u].size(); k++) {
103                        int id = g[u][k];
104                        int v = e[id].v;
105                        ll cap = e[id].cap, w = e[id].cost;
106                        if (d[u] > d[u] + w && cap > 0) {
107                            d[v] = d[u] + w;
108                            relax = true;
109                        }
110                    }
111                }
112            }
113            for(int i = 0; i < n; i++) if(d[i] < INF) potential[i] = d[i];
114        }
115        while (flow < goal && dijkstra()) flow += send_flow(t, goal -
116            flow);
117        flow_through.assign(mxid + 10, 0);
118        for (int u = 0; u < n; u++){
119            for (auto v : g[u])
120                if (e[v].id >= 0) flow_through[e[v].id] = e[v ^ 1].cap;
121        }
122        return make_pair(flow, cost);
123    }
124 };

```

3.17 Prims

```

1 vector<bool> vis(n);
2 priority_queue<pii, vector<pii>, greater<pii>> q;
3
4 auto process = [&](int u){
5     vis[u] = 1;
6     for(int i=0;i<sz(adj[u]);i++){
7         pii v = adj[u][i];
8         swap(v.S,v.F);
9         q.push(v);
10    }
11 }
12 process(0);
13
14 while(!q.empty()){
15     auto [w,u] = q.top();q.pop();
16     if(!vis[u]){cost += w;process(u);}
17 }

```

3.18 SCC Kosaraju

```

1 class SCC{
2 private:
3     int n;
4     vector<vi> adj;
5     vector<vi> adj_t;
6     vector<vi> adj_cond;
7     vi order;
8     vector<bool> vis;
9     vi who;
10    int comp;
11 public:
12    SCC(int n,vector<vi> &a,vector<vi>&b):n(n),adj(a),adj_t(b){
13        comp = -1;
14        who.resize(n);

```

```

15 vis.resize(n);
16 for(int i=0;i<n;i++)
17     if(!vis[i])dfs1(i);
18
19 vis.assign(n,0);
20 for(int i=n-1;i>=0;i--)
21     if(!vis[order[i]]){
22         comp++;
23         dfs2(order[i]);
24     }
25 adj_cond.resize(comp+1);
26 for(int u=0;u<n;u++)
27     for(auto v:adj[u])
28         if(who[v]!=who[u])
29             adj_cond[who[u]].pb(who[v]);
30 }
31
32 void dfs1(int u) {
33     vis[u] = true;
34     for (int v:adj[u])
35         if (!vis[v])dfs1(v);
36     order.pb(u);
37 }
38
39 void dfs2(int u) {
40     vis[u] = 1;
41     for (int v : adj_t[u])
42         if (!vis[v])dfs2(v);
43     who[u] = comp;
44 }
45 };

```

3.19 State Space Graph

```

1 vector<vll> dist(n,vll(1001,INF64));
2 vector<vector<bool>> vis(n,vector<bool> (1001,0));
3 typedef array<ll,3> T;
4 priority_queue<T ,vector<T>,greater<T>> pq;
5 pq.push({0,0,state[0]});
6 dist[0][state[0]]= 0;
7 while(!pq.empty()){
8     auto [d,u,s] = pq.top();
9     pq.pop();
10    if(vis[u][s])continue;
11    vis[u][s] = 1;
12    for(auto [v,w]:adj[u]){
13        ll c = min(s,slow[v]);
14        //Nuevo estad cuando llegue a V
15        if(dist[u][s]+w*s<dist[v][c]){
16            dist[v][c] = dist[u][s]+w*s;
17            pq.push({dist[v][c],v,c});
18        }
19    }
20 }

```

3.20 Topological Sort

```

1 vi ts;
2 queue<int> q;
3 for (int i = 1; i < n+1; i++)
4     if(!in[i])q.push(i);
5
6 while (!q.empty()) {
7     int u=q.front();
8     q.pop();
9     ts.pb(u);
10    for (int v : adj[u]) {
11        in[v]--;

```

```

12        if(!in[v])q.push(i);
13    }
14 }
15
16 if(sz(ts)!=n)cout <<"IMPOSSIBLE";

```

3.21 Virtual Tree

```

1 //T[x] tiempo entrada
2 auto virtualTree = [&](vi &v)->vi{
3     sort(all(v), [&](int x,int y){
4         return T[x] < T[y];
5     });
6     int aux = sz(v)-1;
7     for(int i=0;i<aux;i++)
8         v.pb(lca(v[i],v[i+1]));
9
10    sort(all(v), [&](int x,int y){
11        return T[x] < T[y];
12    });
13    v.erase(unique(all(v)),v.end());
14
15    stack<int> st;
16    st.push(v[0]);
17    for(int i=1;i<sz(v);i++){
18        while(!st.empty() && !isanc(v[i],st.top()))st.pop();
19        tree[st.top()].push_back(v[i]);
20        st.push(v[i]);
21    }
22    return v;
23 };

```

4 Trees

4.1 Centroid Decomposition

```

1 vector<bool> removed(n+1);
2 vi sub(n+1);
3
4 auto dfs = [&](auto &self,int u,int p)->void{
5     sub[u] = 1;
6     for(int v:adj[u]){
7         if(v!=p && !removed[v]){
8             self(self,v,u);
9             sub[u] += sub[v];
10        }
11    }
12 };
13
14 auto centroid = [&](auto &self,int u,int p,int root)->int{
15     for(int v:adj[u]){
16         if(v!=p && !removed[v]){
17             if(sub[v]*2 > sub[root])
18                 return self(self,v,u,root);
19        }
20    }
21    return u;
22 };
23
24 auto info = [&](auto &self,int u,int p,int d,vi &aux)->void{
25     d ^= a[u-1];
26     aux.pb(d);
27     for(int v:adj[u])
28         if(v!=p && !removed[v])
29             self(self,v,u,d,aux);
30 };
31
32 auto decompose = [&](auto &self,int u,int p)->void{

```

```

33     dfs(dfs,u,p);
34     int c = centroid(centroid,u,p,u);
35     //get info
36     removed[c] = 1;
37     for(int v:adj[c]){
38         if(v!=p && !removed[v])
39             self(self,v,c);
40     }
41 };

```

4.2 Tree Diameter

```

1 vector<vector<pii>> adj;
2 vector<bool> vis;
3
4 int bfs(int n,vll &d,int v = 0){
5     vis.assign(n,0);
6     d.assign(n,0);
7     d[v] = 0;
8     queue<int> q;
9     q.push(v);
10    pll last = {v,0};
11    vis[v] = 1;
12    while(!q.empty()){
13        int u = q.front();q.pop();
14        for(auto [w,c]:adj[u]){
15            if(vis[w])continue;
16            d[w] = d[u]+c;
17            q.push(w);
18            vis[w] = 1;
19            if(d[w]>last.S)last = {w,d[w]};
20        }
21    }
22    return int(last.F);
23 }
24
25 void solve(){
26     int n;cin >> n;
27     adj.assign(n,vector<pii> ());
28     for(int i=1;i<n;i++){
29         int a,b;ll c;
30         cin >> a >> b >> c;
31         adj[--a].pb({--b,c});
32         adj[b].pb({a,c});
33     }
34     vll dx(n),dy(n);
35     int a =bfs(n,dx);
36     int b =bfs(n,dy,a);
37     bfs(n,dx,b);
38
39     for(int i=0;i<n;i++){
40         cout << max(dx[i],dy[i]) << " \n"[i==(n-1)];
41     }
42
43 }

```

4.3 Euler Tour

```

1 int timer = 0;
2 vi start,final;
3 vector<vi> adj;
4 void dfs(int u = 0,int prev = -1){
5     start[u]=timer++;
6     for(int v:adj[u]){
7         if(v!=prev)dfs(v,u);
8     }
9     final[u] = timer;
10 }
11 // [start,final)

```

4.4 Heavy Light Decomposition

```

1 class HLD{
2 private:
3     int n;
4     vector<vi> adj;
5     vi parent;
6     vi heavy;
7     vi depth;
8     vi root;
9     vi treePos;
10    Segment_Tree tree;
11
12    int dfs(int u=0){
13        int size = 1, mx_sub = 0;
14        for(auto v:adj[u])
15            if(v!=parent[u]){
16                parent[v] = u;
17                depth[v] = depth[u]+1;
18                int sub = dfs(v);
19                if(sub > mx_sub){
20                    heavy[u] = v;
21                    mx_sub = sub;
22                }
23                size += sub;
24            }
25        return size;
26    }
27
28    template <class BinaryOperation>
29    void processPath(int u, int v, BinaryOperation op) {
30        for (; root[u] != root[v]; v = parent[root[v]]) {
31            if (depth[root[u]] > depth[root[v]]) swap(u, v);
32            op(treePos[root[v]], treePos[v] + 1);
33        }
34        if (depth[u] > depth[v]) swap(u, v);
35        op(treePos[u], treePos[v] + 1);
36    }
37
38    void init(vi &a){
39        for(int i=0,crt=0;i<n;++i){
40            if(parent[i] == -1 || heavy[parent[i]]!=i){
41                for(int j = i;j!=-1;j = heavy[j]){
42                    root[j] = i;
43                    treePos[j] = crt++;
44                }
45            }
46        }
47        tree = Segment_Tree(n);
48        for(int i=0;i<n;i++){
49            tree.update(treePos[i],a[i]);
50        }
51    }
52
53 public:
54
55    HLD(int n,vector<vi> &vale,vi &a):n(n),adj(vale){
56        parent.assign(n,0);
57        heavy.assign(n,-1);
58        depth.assign(n,0);
59        root.resize(n);
60        treePos.assign(n,0);
61        parent[0] = -1;
62        dfs();
63        init(a);
64    }
65

```

```

66    void update(int u,int &val){
67        tree.update(treePos[u],val);
68    }
69
70    void updatePath(int u, int v, int value) {
71        processPath(u, v, [this, &value](int l, int r) {
72            tree.update_range(l, r, value);
73        });
74    }
75
76    int query(int u,int v){
77        int ans = 0;
78        processPath(u,v,[this,&ans](int l,int r){
79            ans = max(ans,tree.query(l,r));
80        });
81        return ans;
82    }
83 };

```

4.5 LCA

```

1 const int LG = 18;
2
3 vi deep(n+1);
4 vector<vi> par(n+1,vi(LG+1));
5
6 auto dfs = [&](auto &self,int u,int p)->void{
7     par[u][0] = p;
8     deep[u] = deep[p]+1;//cost[u] += cost[p]
9     for(int i=1;i<=LG;i++)par[u][i]=par[par[u][i-1]][i-1];
10    for(int v:adj[u])
11        if(v!=p)
12            self(self,v,u);
13 };
14
15 auto lca = [&](int u,int v)->int{
16     if(deep[u] < deep[v])swap(u,v);
17     for(int k=LG;k>=0;k--){
18         if(deep[par[u][k]] >= deep[v])
19             u = par[u][k];
20     }
21     for(int k=LG;k>=0;k--){
22         if(par[u][k]!=par[v][k])
23             u=par[u][k],v=par[v][k];
24     }
25     return par[u][0];
26 };
27
28 auto dist = [&](int u,int v)->int{
29     int lc = lca(u,v);
30     return deep[u]+deep[v]-(deep[lc]<<1);
31 };
32
33 auto kth = [&](int u, int k)->int{
34     assert(k>=0);
35     for(int i=0;i<=LG;i++){
36         if(k&(1<<i))u = par[u][i];
37     }
38     return u;
39 }
40
41 auto isanc = [&](int u,int g)->bool{
42     int k = deep[u]-deep[g];
43     return k >= 0 && kth(u, k) == g;
44 };

```

4.6 Min/Max Weight on Path

```

1 const int MAX = 1e5+5, LG=18;
2 vector<vector<pii>> adj(MAX,vector<pii> ());

```

```

3 vi deep(MAX);
4 int par[MAX][LG+1];
5 int mn[MAX][LG+1];
6 int mx[MAX][LG+1];
7
8 void dfs(int u=1,int p=0,int costmn = INF,int costmx = -INF){
9     par[u][0] = p;
10    deep[u] = deep[p]+1;
11
12    if(p!=0){
13        mn[u][0] = costmn;
14        mx[u][0] = costmx;
15    }
16    for(int i=1;i<=LG;i++){
17        par[u][i]=par[par[u][i-1]][i-1];
18        mn[u][i]=min(mn[u][i-1],mn[par[u][i-1]][i-1]);
19        mx[u][i]=max(mx[u][i-1],mx[par[u][i-1]][i-1]);
20    }
21
22    for(auto [v,t]:adj[u]){
23        if(v!=p){
24            dfs(v,u,t,t);
25        }
26    }
27 }
28
29 pii minmax(int u,int v){
30     pii ans = {INF,-INF};
31     if(deep[u] < deep[v])swap(u,v);
32     for(int k=LG;k>=0;k--){
33         if(deep[par[u][k]] >= deep[v]){
34             ans.F = min(ans.F,mn[u][k]);
35             ans.S = max(ans.S,mx[u][k]);
36             u = par[u][k];
37         }
38         if(u==v)return ans;
39         for(int k=LG;k>=0;k--){
40             if(par[u][k]!=par[v][k]){
41                 ans.F = min({ans.F,mn[u][k],mn[v][k]});
42                 ans.S = max({ans.S,mx[u][k],mx[v][k]});
43                 u=par[u][k],v=par[v][k];
44             }
45         }
46         ans.F = min({ans.F,mn[u][0],mn[v][0]});
47         ans.S = max({ans.S,mx[u][0],mx[v][0]});
48     }
49     return ans;
50 }

```

4.7 Intersection Path

```

1 pair<int, int> getCommonPath(int u, int a, int v, int b) {
2     // returns common path between u->a and v->b where u is ancestor a
3     // and v is ancestor b
4     if (!isanc(a, v)) return make_pair(0, 0);
5     int x = lca(a, b);
6     if (dep[v] < dep[u]) {
7         if (isanc(x, u))
8             return make_pair(u, x);
9         }else{
10            if (isanc(x, v))
11                return make_pair(v, x); // v->x is the common path
12        }
13    }
14    return make_pair(0, 0);
15 }
16
17 pair<int, int> path_union_light(pair<int, int> x, pair<int, int> y) {
18     if (x.first == 0) return y; //where {0,0} is a null path
19     if (y.first == 0) return x;
20     vector<int> can = {x.first, x.second, y.first, y.second};
21     int a = can[0];
22 }

```

```

19 for (int u : can) if (dep[u] > dep[a]) a = u;
20 int b = -1;
21 for (int u : can) if (!isanc(a, u)) {
22     if (b == -1) b = u;
23     if (dep[b] < dep[u]) b = u;
24 }
25 if (b == -1) {
26     b = can[0];
27     for (int u : can) if (dep[u] < dep[b]) b = u;
28     return {a, b};
29 }
30 return {a, b};
31 }
32 // returns the intersection of two paths (a,b) and (c,d)
33 // {0,0} for null path
34 pair<int, int> path_intersection(int a, int b, int c, int d) {
35     if (a == 0 || b == 0) return {0, 0};
36     int u = lca(a, b), v = lca(c, d); // splits both paths in two
37     // chains from lca to one node
38     pair<int, int> x, y;
39     x = getCommonPath(u, a, v, c);
40     y = getCommonPath(u, a, v, d);
41     pair<int, int> a1 = path_union_light(x, y);
42     x = getCommonPath(u, b, v, c);
43     y = getCommonPath(u, b, v, d);
44     pair<int, int> a2 = path_union_light(x, y);
45     return path_union_light(a1, a2);
46 }

```

4.8 Union Path

```

1 pair<int, int> merge(pair<int, int> x, pair<int, int> y) {
2     if (x.first == 0) return y; //where {0,0} is a null path
3     if (y.first == 0) return x;
4     if (x.first == -1 || y.first == -1) return { -1, -1};
5     vector<int> can = {x.first, x.second, y.first, y.second};
6     int a = can[0];
7     for (int u : can)
8         if (dep[u] > dep[a])
9             a = u;
10    int b = -1;
11    for (int u : can)
12        if (!isanc(a, u)) {
13            if (b == -1) b = u;
14            if (dep[b] < dep[u]) b = u;
15        }
16    if (b == -1) {
17        b = can[0];
18        for (int u : can)
19            if (dep[u] < dep[b])
20                b = u;
21        return {a, b};
22    }
23    int g = lca(a, b);
24    for (int u : can) {
25        if (u == a || u == b) continue;
26        if (dep[u] < dep[g] || (!isanc(a, u) && !isanc(b, u))) return {
27            -1, -1};
28    }
29    return {a, b};
30 }

```

4.9 Prufer Code Build

```

1 auto dfs = [&](auto &self, int u) {
2     for (int v : adj[u]) {
3         if (v != parent[u]) {
4             parent[v] = u;

```

```

5         dfs(v);
6     }
7 }
8 };
9
10 parent[n] = -1;
11 dfs(dfs, n);
12
13 int ptr = -1;
14 vi degree(n);
15 for (int i = 1; i <= n; i++) {
16     degree[i] = sz(adj[i]);
17     if (degree[i] == 1 && ptr == -1)
18         ptr = i;
19 }
20
21 vi code(n - 2);
22 int leaf = ptr;
23 for (int i = 0; i < n - 2; i++) {
24     int next = parent[leaf];
25     code[i] = next;
26     if (--degree[next] == 1 && next < ptr) {
27         leaf = next;
28     } else {
29         ptr++;
30         while (degree[ptr] != 1)
31             ptr++;
32         leaf = ptr;
33     }
34 }

```

4.10 Prufer Code Restore

```

1 int n = code.size() + 2;
2 vi degree(n, 0);
3 for (int i : code)
4     degree[i]++;
5
6 int ptr = 0;
7 while (degree[ptr] != 1)
8     ptr++;
9
10 int leaf = ptr;
11
12 vector<pii> edges;
13 for (int v : code) {
14     edges.emplace_back(leaf, v);
15     if (--degree[v] == 0 && v < ptr) {
16         leaf = v;
17     } else {
18         ptr++;
19         while (degree[ptr] != 1)
20             ptr++;
21         leaf = ptr;
22     }
23 }
24
25 edges.emplace_back(leaf, n - 1);

```

4.11 Tree Rerooting

```

1 namespace reroot {
2     const auto exclusive = [](const auto& a, const auto& base, const
3         auto& merge_into, int vertex) {
4         int n = (int)a.size();
5         using Aggregate = decay_t<decltype(base)>;
6         vector<Aggregate> b(n, base);
7         for (int bit = (int)lg(n); bit >= 0; --bit) {

```

```

7         for (int i = n - 1; i >= 0; --i) b[i] = b[i] >> 1;
8         int sz = n - (n & 1);
9         for (int i = 0; i < sz; ++i) {
10             int index = (i >> bit) ^ 1;
11             b[index] = merge_into(b[index], a[i], vertex, i);
12         }
13     }
14     return b;
15 };
16 // MergeInto : Aggregate * Value * Vertex(int) * EdgeIndex(int)
17 // -> Aggregate
18 // Base : Vertex(int) -> Aggregate
19 // FinalizeMerge : Aggregate * Vertex(int) * EdgeIndex(int) ->
20 // Value
21 const auto rerooter = [](const auto& g, const auto& base, const
22     auto& merge_into, const auto& finalize_merge) {
23     int n = (int)g.size();
24     using Aggregate = decay_t<decltype(base(0))>;
25     using Value = decay_t<decltype(finalize_merge(base(0), 0, 0))>;
26     vector<Value> root_dp(n), dp(n);
27     vector<vector<Value>> edge_dp(n), redge_dp(n);
28
29     vector<int> bfs, parent(n);
30     bfs.reserve(n);
31     bfs.push_back(0);
32     for (int i = 0; i < n; ++i) {
33         int u = bfs[i];
34         for (auto v : g[u]) {
35             if (parent[u] == v) continue;
36             parent[v] = u;
37             bfs.push_back(v);
38         }
39     }
40
41     for (int i = n - 1; i >= 0; --i) {
42         int u = bfs[i];
43         int p_edge_index = -1;
44         Aggregate aggregate = base(u);
45         for (int edge_index = 0; edge_index < (int)g[u].size(); ++
46             edge_index) {
47             int v = g[u][edge_index];
48             if (parent[u] == v) {
49                 p_edge_index = edge_index;
50                 continue;
51             }
52             aggregate = merge_into(aggregate, dp[v], u, edge_index);
53         }
54         dp[u] = finalize_merge(aggregate, u, p_edge_index);
55     }
56
57     for (auto u : bfs) {
58         dp[parent[u]] = dp[u];
59         edge_dp[u].reserve(g[u].size());
60         for (auto v : g[u]) edge_dp[u].push_back(dp[v]);
61         auto dp_exclusive = exclusive(edge_dp[u], base(u),
62             merge_into, u);
63         redge_dp[u].reserve(g[u].size());
64         for (int i = 0; i < (int)dp_exclusive.size(); ++i) redge_dp[
65             u].push_back(finalize_merge(dp_exclusive[i], u, i));
66         root_dp[u] = finalize_merge(n > 1 ? merge_into(dp_exclusive
67             [0], edge_dp[u][0], u, 0) : base(u), u, -1);
68         for (int i = 0; i < (int)g[u].size(); ++i) {
69             dp[g[u][i]] = redge_dp[u][i];
70         }
71     }
72 }

```

```

66     return make_tuple(move(root_dp), move(edge_dp), move(ledge_dp)
67 );
68 }
69
70 //in solve()
71 using Aggregate = int;
72 using Value = int;
73
74 auto base = [&](int vertex) -> Aggregate {
75     //base case
76     return ;
77 };
78
79 auto merge_into = [](Aggregate vertex_dp, Value neighbor_dp, int
80 vertex, int edge_index) -> Aggregate {
81     //merge childs
82     return vertex_dp;
83 };
84
85 auto finalize_merge = [&](Aggregate vertex_dp, int vertex, int
86 edge_index) -> Value {
87     //return of dfs
88     return vertex_dp;
89 };
90
91 auto [root_dp, edge_dp, ledge_dp] = reroot::rerooter(adj, base,
92 merge_into, finalize_merge);

```

4.12 Tree Matching

```

1 vector<vll> dp(2,vll(MAX));
2
3 void dfs(ll u = 0, ll p=-1){
4     dp[0][u] = 0;
5     dp[1][u] = -INF;
6
7     for(ll v:adj[u]){
8         if(v == p)continue;
9         dfs(v,u);
10
11         dp[0][u] += max(dp[1][v], dp[0][v]);
12
13         ll opt = min(dp[0][v] - dp[1][v], 0ll);
14
15         dp[1][u] = max(dp[1][u], opt);
16     }
17
18     dp[1][u] += dp[0][u];
19     dp[1][u]++;
20 }

```

4.13 Rooted Tree Hashing

```

1 auto dfs = [](auto &self, int u, int p, vector<vi> &adj) -> pair<ll, ll>{
2     vector<pair<ll, ll>> shash;
3     for(int v:adj[u]){
4         if(v==p)continue;
5         shash.pb(self(self, v, u, adj));
6     }
7     sort(all(shash));
8
9     ll p1 = P1, p2 = P2;
10    pair<ll, ll> hash = {42, 42};
11    for(int i=0; i<sz(shash); i++, (p1*=p1)%=MOD1, (p2*=p2)%=MOD2){
12        (hash.F += shash[i].F*shash[i].F%MOD1+shash[i].F*p1%MOD1+42)%
13        MOD1;

```

```

13        (hash.S += shash[i].S*shash[i].S%MOD2+shash[i].S*p2%MOD2+42)%
14        MOD2;
15    }
16    return hash;
17 };

```

4.14 Tree Isomorphism

```

1 //1-indexed
2
3 auto centers = [&](vector<vi> &adj) -> vi{
4     //Diameters of a tree
5     vll du(n+1), dv(n+1);
6     int v = bfs(du, 1, adj);
7     int u = bfs(dv, v, adj);
8     bfs(du, u, adj);
9     vi ans;
10    for(int i=1; i<=n; i++){
11        if(du[i]+dv[i]==du[v] && du[i]>=du[v]/2 && dv[i]>=du[v]/2)
12            ans.pb(i);
13    }
14    return ans;
15 };
16
17 //in main
18 vi a = centers(tree1);
19 vi b = centers(tree2);
20
21 for(auto x:a)
22     for(auto y:b)
23         //checar isomorfismos si estan rooteados
24         //en x y y

```

5 Number Theory

5.1 Euler Totient

```

1
2 vi phi(N);
3 iota(all(phi), 0);
4
5 for(int i=2; i<N; i++){
6     if(phi[i]==i)
7         for(int j=i; j<N; j+=i)
8             phi[j] -= phi[j]/i;

```

5.2 Mobius Function

```

1 int mob[N];
2
3 mob[1] = 1;
4 for(int i=2; i<N; i++){
5     mob[i]--;
6     for(int j=i+i; j<N; j+=i)
7         mob[j] -= mob[i];
8 }

```

5.3 Chinese Remainder Theorem

```

1 using T = __int128;
2 // ax + by = __gcd(a, b)
3 // returns __gcd(a, b)
4 T extended_euclid(T a, T b, T &x, T &y){
5     T xx = y = 0;
6     T yy = x = 1;
7     while(b){
8         T q = a / b;
9         T t = b; b = a % b; a = t;
10        t = xx; xx = x - q * xx; x = t;

```

```

11     t = yy; yy = y - q * yy; y = t;
12 }
13 return a;
14 }
15 // finds x such that x % m1 = a1, x % m2 = a2. m1 and m2 may not
16 // be coprime
17 // here, x is unique modulo m = lcm(m1, m2). returns (x, m). on
18 // failure, m = -1.
19 pair<T, T> CRT(T a1, T m1, T a2, T m2){
20     T p, q;
21     T g = extended_euclid(m1, m2, p, q);
22     if (a1 % g != a2 % g) return make_pair(0, -1);
23     T m = m1 / g * m2;
24     p = (p*m + m)%m;
25     q = (q*m + m)%m;
26     return make_pair((p * a2 % m * (m1 / g) % m + q * a1 % m * (m2 /
27     g) % m) % m, m);
28 }
29 }

```

5.4 Binpow

```

1 ll binpow(ll a, ll b){
2     a%=MOD;
3     ll ans = 1;
4     while(b){
5         if(b&1) (ans*=a)%=MOD;
6         (a*=a)%=MOD;
7         b>>=1;
8     }
9     return ans;
10 }

```

6 Combinatorics

6.1 nCr

```

1 // Hash: 2e457c
2 const int MAX = 2e6;
3 ll factorial[MAX+1];
4 ll inv[MAX+1]; // #d459d3
5
6 ll binpow(ll a, ll b){
7     ll res = 1;
8     while(b){
9         if(b&1) res = res*a%MOD;
10        a = a*a%MOD;
11        b>>=1;
12    }
13    return res;
14 } // #89684f
15
16 void precalc(){
17     factorial[0]=1;
18     for (ll i = 1; i <= MAX; i++) {
19         factorial[i] = factorial[i-1]*i%MOD;
20     }
21     inv[MAX] = binpow(factorial[MAX], MOD-2);
22     for (ll i = MAX; i >= 1; i--) {
23         inv[i-1] = inv[i]*i%MOD;
24     }
25 } // #c4e465
26
27 ll ncr(ll a, ll b){
28     return factorial[a]*inv[b]%MOD*inv[a-b]%MOD;
29 } // #f2f007

```

7 Math

7.1 FFT


```

1 typedef complex<double> base;
2
3 void fft(vector<base> &p, bool inv = 0) {
4     int n = p.size(), i = 0;
5     for(int j = 1; j < n - 1; ++j) {
6         for(int k = n >> 1; k > (i ^ k); k >= 1);
7         if(j < i) swap(p[i], p[j]);
8     }
9
10    for(int l = 1, m; (m = l << 1) <= n; l <= m) {
11        double ang = 2 * PI / m *(inv?1.0:-1.0);
12        base wn(cos(ang),sin(ang));
13        for(int i = 0; i < n; i += m) {
14            base w(1,0);
15            for(int j = i, k = i + 1; j < k; ++j, w *= wn) {
16                base t = w * p[j + l];
17                p[j + l] = p[j] - t;
18                p[j] = p[j] + t;
19            }
20        }
21    }
22
23    if(inv) for(int i = 0; i < n; ++i) p[i] /= n;
24 }
25
26 vector<ll> multiply(vi &a, vi &b) {
27     int n = a.size(), m = b.size(), t = n + m - 1, sz = 1;
28     while(sz < t) sz <= 1;
29     vector<base> x(sz), y(sz);
30     for(int i = 0; i < n; ++i) x[i] = base(a[i], 0);
31     for(int i = 0; i < m; ++i) y[i] = base(b[i], 0);
32     fft(x), fft(y);
33     for(int i = 0; i < sz; ++i) x[i] *= y[i];
34     fft(x, 1);
35     vector<ll> ret(sz);
36     for(int i = 0; i < sz; ++i) ret[i] = (ll)round(x[i].real());
37     while((int)ret.size() > 1 && ret.back() == 0) ret.pop_back();
38     return ret;
39 }

```

7.2 FWDT

```

1 const int MOD = 1e9+7;
2 const int inv2 = (MOD + 1) >> 1;
3 #define M (1 << 20)
4 #define OR 0
5 #define AND 1
6 #define XOR 2
7
8 int P1[M],P2[M];
9 void wt(int *a, int n, int flag = XOR){
10     if (n == 0) return;
11     int m = n / 2;
12     wt(a, m, flag); wt(a + m, m, flag);
13     for(int i = 0; i < m; i++){
14         int x = a[i], y = a[i + m];
15         if (flag == OR) a[i] = x, a[i + m] = (x + y) % MOD;
16         if (flag == AND) a[i] = (x + y) % MOD, a[i + m] = y;
17         if (flag == XOR) a[i] = (x + y) % MOD, a[i + m] = (x - y + MOD) % MOD;
18     }
19 }
20
21 void iwt(int* a, int n, int flag = XOR) {
22     if (n == 0) return;
23     int m = n / 2;
24     iwt(a, m, flag); iwt(a + m, m, flag);
25     for(int i = 0; i < m; i++){
26         int x = a[i], y = a[i + m];
27         if (flag == OR) a[i] = x, a[i + m] = (y - x + MOD) % MOD;

```

```

27     if (flag == AND) a[i] = (x - y + MOD) % MOD, a[i + m] = y;
28     if (flag == XOR) a[i] = 1LL * (x + y) * inv2 % MOD, a[i + m] = 1LL * (x - y + MOD) * inv2 % MOD; // replace inv2 by >>1 if not required
29 }
30 }
31
32 vector<int> multiply(int n, vector<int> &A, vector<int> &B, int flag = XOR) {
33     assert(__builtin_popcount(n) == 1);
34     A.resize(n); B.resize(n);
35     for(int i = 0; i < n; i++) P1[i] = A[i];
36     for(int i = 0; i < n; i++) P2[i] = B[i];
37     wt(P1, n, flag); wt(P2, n, flag);
38     for (int i = 0; i < n; i++) P1[i] = 1LL * P1[i] * P2[i] % MOD;
39     iwt(P1, n, flag);
40     return vector<int> (P1, P1 + n);
41 }

```

7.3 NTT

```

1 void ntt(vll &a){
2     int n = sz(a), L = 31 - __builtin_clz(n);
3     static vll rt(2, 1);
4     for (static int k = 2, s = 2; k < n; k *= 2, s++) {
5         rt.resize(n);
6         ll z[] = {1, binpow(root, MOD >> s)};
7         for(int i=k;i<2*k;i++)
8             rt[i] = rt[i / 2] * z[i & 1] % MOD;
9     }
10    vi rev(n);
11    for(int i=0;i<n;i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
12    for(int i=0;i<n;i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
13    for (int k = 1; k < n; k *= 2)
14        for (int i = 0; i < n; i += 2 * k)
15            for(int j=0;j<k;j++){
16                ll z = rt[j + k] * a[i + j + k] % MOD, &ai = a[i + j];
17                a[i + j + k] = ai - z + (z > ai ? MOD : 0);
18                ai += (ai + z >= MOD ? z - MOD : z);
19            }
20 }
21
22 vll conv(const vll &a, const vll &b){
23     if (a.empty() || b.empty()) return {};
24     int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
25         n = 1 << B;
26     int inv = binpow(n, MOD - 2);
27     vll L(a), R(b), out(n);
28     L.resize(n), R.resize(n);
29     ntt(L), ntt(R);
30     for(int i=0;i<n;i++)
31         out[-i & (n - 1)] = (ll)L[i] * R[i] % MOD * inv % MOD;
32     ntt(out);
33     return {out.begin(), out.begin() + s};
34 }

```

7.4 FFT Any Mod

```

1 template<int M> vll convMod(const vll &a, const vll &b) {
2     if(a.empty() || b.empty()) return {};
3     vll res(sz(a) + sz(b) - 1);
4     int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut = int(sqrt(M));
5     vector<base> L(n), R(n), outs(n), outl(n);
6     for(int i = 0; i < sz(a); i++) L[i] = base((int)a[i] / cut, (int)a[i] % cut);
7     for(int i = 0; i < sz(b); i++) R[i] = base((int)b[i] / cut, (int)b[i] % cut);

```

```

8     fft(L), fft(R);
9     for(int i = 0; i < n; i++) {
10         int j = -i & (n - 1);
11         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
12         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
13     }
14     fft(outl), fft(outs);
15     for(int i = 0; i < sz(res); i++) {
16         ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5);
17         ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
18         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
19     }
20     return res;
21 }

```

7.5 Or Convolution

```

1 void solve(){
2     int n,k; cin>>n>>k;
3     vector<ll> acu(1<<k), sos(1<<k);
4     string s;
5     for(int i=0; i<n ; i++){
6         cin>>s;
7         sos[stoi(s,nullptr, 2)]++;
8     }
9     for (int i = 0; i < k; i++) {
10        for (int j = 0; j < (1 << k); j++) {
11            if ((j >> i) & 1) {
12                sos[j] += sos[j - (1 << i)];
13            }
14        }
15    }
16    for(int i=0; i<(1<<k); i++) {
17        if(i<3) acu[i]=0;
18        acu[i] = ((sos[i])*(sos[i]-1)*(sos[i]-2))/6;
19    }
20    for (int i = k - 1; i >= 0; i--) {
21        for (int j = (1 << k) - 1; j >= 0; j--) {
22            if ((j >> i) & 1) {
23                acu[j] -= acu[j - (1 << i)];
24            }
25        }
26    }
27    int m; cin>>m;
28    for(int i=0; i<m; i++){
29        cin>>s;
30        cout<<acu[stoi(s,nullptr, 2)]<<endl;
31    }
32 }

```

7.6 Xor Basis

```

1 template<typename T = int, int B = 31>
2 struct Basis{
3     T basis[B]; int sz;
4     void clear() {
5         memset(basis, 0, sizeof basis);
6         sz = 0;
7     }
8     Basis(){clear();}
9
10    void insert(T x) {
11        for(int i=B-1;i>=0;i--){
12            if(x>>i&1){
13                if(basis[i])x ^= basis[i];
14                else{
15                    basis[i] = x;
16                    sz++;

```

```

17         break;
18     }
19 }
20 }
21 }
22
23 bool can(T x){
24     for(int i=B-1;i>=0;i--){
25         x = min(x, x ^ basis[i]);
26         return x == 0;
27     }
28
29 T kth(T k){
30     T x = 0;
31     T cnt = ((T)1<<sz);
32     for(int i=B-1;i>=0;i--){
33         if(!basis[i])continue;
34         if(k>(cnt>>1)){
35             if (!(x>>i&1))x ^= basis[i];
36             k -= (cnt>>1);
37         }else{
38             if (x>>i&1)x ^= basis[i];
39         }
40         cnt>>=1;
41     }
42     return x;
43 }
44
45 bool merge(Basis &b){
46     bool flag = false;
47     for(ll i=B-1;i>=0;i--){
48         if(b.basis[i] && !insert(b.basis[i])) flag = true;
49     }
50     return flag;
51 }
52 };

```

7.7 Matrix

```

1 struct Mat {
2     int n, m;
3     vector<vector<int>> &a;
4     Mat() { }
5     Mat(int _n, int _m) { n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
6     Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0; a = v; }
7     inline void make_unit() {
8         assert(n == m);
9         for (int i = 0; i < n; i++) {
10             for (int j = 0; j < n; j++) a[i][j] = i == j;
11         }
12     }
13     inline Mat operator + (const Mat &b) {
14         assert(n == b.n && m == b.m);
15         Mat ans = Mat(n, m);
16         for(int i = 0; i < n; i++) {
17             for(int j = 0; j < m; j++) {
18                 ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
19             }
20         }
21         return ans;
22     }
23     inline Mat operator - (const Mat &b) {
24         assert(n == b.n && m == b.m);
25         Mat ans = Mat(n, m);
26         for(int i = 0; i < n; i++) {
27             for(int j = 0; j < m; j++) {

```

```

28         ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
29     }
30 }
31     return ans;
32 }
33     inline Mat operator * (const Mat &b) {
34         assert(m == b.n);
35         Mat ans = Mat(n, b.m);
36         for(int i = 0; i < n; i++) {
37             for(int j = 0; j < b.m; j++) {
38                 for(int k = 0; k < m; k++) {
39                     ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] %
40                         mod) % mod;
41                 }
42             }
43             return ans;
44         }
45     inline Mat pow(long long k) {
46         assert(n == m);
47         Mat ans(n, n), t = a; ans.make_unit();
48         while (k) {
49             if (k & 1) ans = ans * t;
50             t = t * t;
51             k >>= 1;
52         }
53         return ans;
54     }
55     inline Mat& operator += (const Mat& b) { return *this = (*this)
56         + b; }
57     inline Mat& operator -= (const Mat& b) { return *this = (*this)
58         - b; }
59     inline Mat& operator *= (const Mat& b) { return *this = (*this)
60         * b; }
61     inline bool operator == (const Mat& b) { return a == b.a; }
62     inline bool operator != (const Mat& b) { return a != b.a; }
63 };
64
65 // Usage
66 // Mat a(n, n);
67 // Mat b(n, n);
68 // Mat c = a * b;
69 // Mat d = a + b;
70 // Mat e = a - b;
71 // Mat f = a.pow(k);
72 // a.a[i][j] = x;

```

8 Strings

8.1 Phi Function

```

1 vi pi(n,0);
2 for(int i=1;i<n;i++){
3     int j = pi[i-1];
4     while(j && s[i]!=s[j])j = pi[j-1];
5     if(s[i] == s[j])j++;
6     pi[i] = j;
7 }

```

8.2 Z Function

```

1 // greatest number of characters starting from pos i
2 // that coincide with the first characters of s
3
4 vi z(n,0);
5 int l = 0, r = 0;
6 for(int i=1;i<n;i++){
7     if(i<r)z[i] = min(r-i,z[i-l]);

```

```

8
9     while(i+z[i]<n && s[z[i]] == s[i+z[i]])z[i]++;
10
11     if(i+z[i]>r){
12         l = i;
13         r = i+z[i];
14     }
15 }

```

8.3 Aho Corasick

```

1 struct AC{
2     static const int K = 26;
3     struct Node{
4         int ch[K];
5         int fail;
6         int last_m;
7         vi id;
8         Node(){
9             memset(ch,0,sizeof(ch));
10            fail = last_m = 0;
11        }
12    };
13
14    vector<Node> t;
15
16    AC(){t.emplace_back();}
17
18    void insert(string& s, int id){
19        int u = 0;
20        for(char c:s){
21            int v = c-'a';
22            if(!t[u].ch[v]){
23                t[u].ch[v] = sz(t);
24                t.emplace_back();
25            }
26            u = t[u].ch[v];
27        }
28        t[u].id.push_back(id);
29    }
30
31    void build(){
32        queue<int> q;
33        for(int i=0;i<K;i++)
34            if (t[0].ch[i]) q.push(t[0].ch[i]);
35
36        while(!q.empty()){
37            int u=q.front();q.pop();
38            int f = t[u].fail;
39            t[u].last_m = t[f].id.empty() ? t[f].last_m:f;
40
41            for(int i=0;i<K;i++){
42                int &v = t[u].ch[i];
43                if(v){
44                    t[v].fail = t[f].ch[i];
45                    q.push(v);
46                }else v = t[f].ch[i];
47            }
48        }
49    }
50
51    int count(string &s){
52        ll ans = 0;
53        int u = 0;
54        for(char c:s){
55            u = t[u].ch[c-'a'];
56            int temp = u;
57            while(temp > 0){

```

```

58     ans += t[temp].id.size();
59     temp = t[temp].last_m;
60 }
61 }
62 return ans;
63 }
64 };

```

8.4 Dynamic Aho Corasick

```

1 vector<AC> a(19); // log aho corasick
2 vector<vector<string>> ws;
3 int cnt = 0; // number of strings insert
4
5 auto insert = [&](string &s){
6     int pos = 0;
7     for(int i=0; i<19; i++){
8         if(ws[i].empty()){
9             pos = i;
10            break;
11        }
12    }
13    ws[pos].pb(s);
14    for(int i=0; i<pos; i++){
15        for(auto &str:ws[i])
16            ws[pos].pb(move(str));
17        ws[i].clear();
18        w[i] = AC();
19    }
20
21    w[pos] = AC();
22
23    for(int i=0; i<sz(ws[pos]); i++) w[pos].insert(ws[pos][i], i+1);
24
25    w[pos].build();
26
27 };
28
29 auto query = [&](string &s)->ll{
30     ll ans = 0;
31     for(int i=0; i<19; i++){
32         if(cnt>>i&1)
33             ans += a[i].count(s);
34     };

```

8.5 Suffix Array

```

1 struct SuffixArray{
2     vi sa, lcp;
3     SuffixArray(string s, int lim=256) { // or vector<int>
4         s.push_back(0); int n = sz(s), k = 0, a, b;
5         vi x(all(s)), y(n), ws(max(n, lim));
6         sa = lcp = y, iota(all(sa), 0);
7         for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
8             p = j, iota(all(y), n - j);
9             for(int i=0; i<n; i++) if (sa[i] >= j) y[p++] = sa[i] - j;
10            fill(all(ws), 0);
11            for(int i=0; i<n; i++) ws[x[i]]++;
12            for(int i=1; i<lim; i++) ws[i] += ws[i - 1];
13            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
14            swap(x, y), p = 1, x[sa[0]] = 0;
15            for(int i=1; i<n; i++){
16                a = sa[i - 1];
17                b = sa[i];
18                x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p
19                ++;
20            }
21        }

```

```

21     for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
22         for (k && k--, j = sa[x[i] - 1];
23             s[i + k] == s[j + k]; k++);
24     }
25 };

```

8.6 Hashing

```

1 ll binpow(ll n, ll k, const int mod){
2     ll res = 1;
3     n%=mod;
4     while(k){
5         if(k&1)(res*=n)%=mod;
6         (n*=n)%=mod;
7         k>>=1;
8     }
9     return res;
10 }
11
12 const int MOD1 = 127657753, MOD2 = 987654319;
13 const int P1 = 137, P2 = 277;
14 const int MAX = 1e6+9;
15
16 vector<pll> P(MAX), I_P(MAX);
17
18 void calc(){
19     P[0] = {1, 1};
20     for(int i=1; i<MAX; i++){
21         P[i].F = P[i-1].F*P1%MOD1;
22         P[i].S = P[i-1].S*P2%MOD2;
23     }
24     int i1 = binpow(P1, MOD1-2, MOD1);
25     int i2 = binpow(P2, MOD2-2, MOD2);
26     I_P[0] = {1, 1};
27     for(int i=1; i<MAX; i++){
28         I_P[i].F = I_P[i-1].F*i1%MOD1;
29         I_P[i].S = I_P[i-1].S*i2%MOD2;
30     }
31 }
32
33 struct Hashing {
34     int n;
35     string s; // 0 - indexed
36     vector<pll> hs; // 1 - indexed
37
38     Hashing(string _s) {
39         n = _s.size();
40         s = _s;
41         hs.emplace_back(0, 0);
42         for (int i = 0; i < n; i++) {
43             pll p;
44             p.F = (hs[i].F + P[i].F * (s[i] - 'a' + 1) % MOD1) % MOD1;
45             p.S = (hs[i].S + P[i].S * (s[i] - 'a' + 1) % MOD2) % MOD2;
46             hs.push_back(p);
47         }
48     }
49
50     pll get_hash(int l, int r) { // 1 - indexed
51         pll ans;
52         ans.F = (hs[r].F - hs[l - 1].F + MOD1) * iLL * I_P[l - 1].F%
53             MOD1;
54         ans.S = (hs[r].S - hs[l - 1].S + MOD2) * iLL * I_P[l - 1].S%
55             MOD2;
56         return ans;
57     }
58 };

```

8.7 Palindromic Tree

```

1 struct Eertree{
2     struct Node{
3         int nxt[26], len, st, en, link, cnt, oc;
4     }; //f0c5da
5
6     vector<Node> t;
7     string s;
8     int sz, last;
9     Eertree(){} //24d5fc
10
11     Eertree(string _s){
12         s = _s;
13         int n = sz(s);
14         t.resize(n+5);
15         sz = 2, last = 2;
16         t[1].len = -1, t[1].link = 1;
17         t[2].len = 0, t[2].link = 1;
18     } //1fe603
19
20     int get_link(int u, int pos){
21         while(pos-1-t[u].len<0 || s[pos-1-t[u].len] != s[pos])
22             u = t[u].link;
23         return u;
24     } //72a63c
25
26     bool add(int pos){
27         int ch = s[pos] - 'a';
28         int cur = get_link(last, pos);
29
30         if(t[cur].nxt[ch]){
31             last = t[cur].nxt[ch];
32             t[last].oc++;
33             return 0;
34         } //4c7cf1
35
36         sz++;
37         t[sz].len = t[cur].len+2;
38         t[sz].en = pos;
39         t[sz].st = pos-t[sz].len+1; //3b0156
40
41         if(t[sz].len == 1) t[sz].link = 2;
42         else t[sz].link = t[get_link(t[cur].link, pos)].nxt[ch];
43
44         t[sz].cnt = t[t[sz].link].cnt+1;
45         t[sz].oc = 1;
46         t[cur].nxt[ch] = sz;
47         last = sz;
48         return 1; //e1a942
49     } //4b0fbf
50
51     void calc_occurrences(){
52         for(int i=sz; i>2; i--){
53             t[t[i].link].oc += t[i].oc;
54         } //38a897
55
56     void show(){
57         for(int i=3; i<=sz; i++) cout << s.substr(t[i].st, t[i].len) << '\n';
58     } //996a3b
59
60 }; //5bf93b

```

8.8 Manacher

```

1 struct Manacher{
2     vector<int> p[2];
3     // p[1][i] = (max odd length palindrome centered at i) / 2 [
4         floor division]

```

```

4 // p[0][i] = same for even, it considers the right center
5 // e.g. for s = "abbabba", p[1][3] = 3, p[0][2] = 2
6 Manacher(string &s){
7     int n = sz(s);
8     p[0].resize(n+1);
9     p[1].resize(n);
10    for (int z=0;z<2;z++){
11        for(int i=0,l=0,r=0;i<n;i++){
12            int t = r-i+l;
13            if(i<r)p[z][i] = min(t,p[z][l+t]);
14            int L=i- p[z][i],R=i+p[z][i]-l;
15            while (L>=1&&R+1<n && s[L-1] == s[R+1])
16                p[z][i]++, L--, R++;
17            if(R>r)l=L,r=R;
18        }
19    }
20 }

21 bool is_palindrome(int l, int r){
22     int mid = (l+r+1)/2,len = r-l+1;
23     return 2 * p[len % 2][mid] + len % 2 >= len;
24 }
25 }
26 };

```

9 Dp

9.1 Convex Hull Trick

```

1 //Hash: #d1459e
2 struct CHT {
3     vector<ll> m, b;
4     int ptr = 0;
5
6     bool bad(int l1, int l2, int l3) {
7         return 1.0*(b[l3]-b[l1])*(m[l1]-m[l2])<=1.0*(b[l2]-b[l1])*(m[
8             l1]-m[l3]); //(slope dec+query min), (slope inc+query max) // #
9             aaf1b6
10            return 1.0*(b[l3]-b[l1])*(m[l1]-m[l2]) >1.0*(b[l2]-b[l1])*(m[
11                l1]-m[l3]); //(slope dec+query max), (slope inc+query min) //
12                #c3130b
13        }
14
15        void add(ll _m, ll _b){
16            m.push_back(_m);
17            b.push_back(_b);
18            int s = m.size();
19            while(s>=3&&bad(s-3, s-2, s-1)){
20                s--;
21                m.erase(m.end()-2);
22                b.erase(b.end()-2);
23            }
24        } // #b5b4ed
25
26        ll f(int i, ll x){return m[i] * x + b[i];} // #d77a99
27
28        //(slope dec+query min), (slope inc+query max) -> x increasing
29        //(slope dec+query max), (slope inc+query min) -> x decreasing
30        ll query(ll x){
31            if(ptr >= sz(m))ptr = m.size() - 1;
32            while(ptr < sz(m)-1 && f(ptr+1, x)<f(ptr, x))ptr++;
33            return f(ptr, x);
34        } // #19a165
35
36        ll bs(int l, int r, ll x){
37            int mid = (l + r) / 2;
38            if(mid + 1 < m.size() && f(mid + 1, x) < f(mid, x)) return bs(
39                mid + 1, r, x); // > for max

```

```

35     if(mid - 1 >= 0 && f(mid - 1, x) < f(mid, x)) return bs(l, mid
36         - 1, x); // > for max
37     return f(mid, x);
38 } // #59c8d0
39 };

```

9.2 Knuth Division

```

1 int N;
2 // read N and input
3 int dp[N][N], opt[N][N];
4
5 auto C = [&](int i, int j) {
6     // Implement cost function C.
7 };
8
9 for (int i=0;i<N;i++){
10     opt[i][i] = i;
11     // Initialize dp[i][i] according to the problem
12 }
13
14 for (int i = N-2; i >= 0; i--){
15     for (int j = i+1; j < N; j++){
16         int mn = INT_MAX;
17         int cost = C(i, j);
18         for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
19             if (mn >= dp[i][k] + dp[k+1][j] + cost) {
20                 opt[i][j] = k;
21                 mn = dp[i][k] + dp[k+1][j] + cost;
22             }
23         }
24         dp[i][j] = mn;
25     }
26 } // #09b41e
27 ans = dp[0][N-1];

```

9.3 Li Chao Tree

```

1 typedef ll ftype;
2 typedef complex<ftype> point;
3 #define x real
4 #define y imag
5 const ll MOD=1e9+7; //998244353LL;
6 const ll INF=1LL<<60;
7 const int maxn = 2e5;
8 point line[4 * maxn];
9 ftype dot(point a, point b) {
10     return (conj(a) * b).x();
11 }
12 ftype f(point a, ftype x) {
13     return dot(a, {x, 1});
14 }
15 void add_line(point nw, int v, int l, int r, int
16     lf, int rf) {
17     int m = (l + r) / 2;
18     if(rf < l || r <= lf) return;
19     if(l < lf || rf < r-1){
20         add_line(nw, 2 * v, l, m, lf, rf);
21         add_line(nw, 2 * v + 1, m, r, lf, rf);
22     }
23     return;
24 }
25 bool lef = f(nw, l) < f(line[v], l);
26 bool mid = f(nw, m) < f(line[v], m);
27 if(!mid) {
28     swap(line[v], nw);
29 }
30 if(r - l == 1) {
31     return;

```

```

31 } else if(lef != mid) {
32     add_line(nw, 2 * v, l, m, lf, rf);
33 } else {
34     add_line(nw, 2 * v + 1, m, r, lf, rf);
35 }
36 }
37 ftype get(int x, int v, int l, int r) {
38     int m = (l + r) / 2;
39     if(r - l == 1) {
40         return f(line[v], x);
41     } else if(x < m) {
42         return max(f(line[v], x), get(x, 2 * v,
43             l, m));
44     } else {
45         return max(f(line[v], x), get(x, 2 * v +
46             1, m, r));
47     }

```

10 Geometry 2D

10.1 Point

```

1 const double eps = 1e-9;
2 int sign(double x){return (x>eps)-(x<-eps);}
3
4 struct PT{
5     double x,y;
6     PT(){x=0,y=0;}
7     PT(double x,double y):x(x),y(y){}
8     PT(const PT &p):x(p.x),y(p.y){}
9     PT operator+(const PT &a)const{return PT(x+a.x,y+a.y);}
10    PT operator-(const PT &a)const{return PT(x-a.x,y-a.y);}
11    PT operator*(const double a)const{return PT(x*a,y*a);}
12    friend PT operator*(const double &a, const PT &b){return PT(a*b.
13        x,a* b.y);}
14    PT operator / (const double a)const{return PT(x/a,y/a);}
15    bool operator == (PT a)const{return sign(a.x-x)==0 && sign(a.y-y
16        )==0;}
17    bool operator != (PT a)const{return !(*this==a);}
18    bool operator < (PT a) const{return sign(a.x-x)==0?y<a.y:x<a.x;}
19    bool operator > (PT a) const{return sign(a.x-x)==0?y>a.y:x>a.x;}
20    double norm(){return sqrt(x*x+y*y);}
21    double norm2(){return x*x+y*y;}
22    PT perp(){return PT(-y,x);}
23    double arg(){return atan2(y,x);}
24
25    // returns a vector with norm r and having same direction
26    PT truncate(double r){
27        double k = norm();
28        if(!sign(k))return *this;
29        r/=k;
30        return PT(x*r,y*r);
31    }
32
33    istream &operator >> (istream &in,PT &p){return in >> p.x >> p.y;}
34    ostream &operator << (ostream &out, PT &p){return out << "(" << p.
35        x << "," << p.y << ")";}
36
37    inline double dot(PT a,PT b){return a.x*b.x+a.y*b.y;}
38    inline double dist2(PT a,PT b){return dot(a-b,a-b);}
39    inline double dist(PT a,PT b){return sqrt(dot(a-b,a-b));}
40    inline double cross(PT a,PT b){return a.x*b.y-a.y*b.x;}
41    inline double cross2(PT a,PT b,PT c){return cross(b-a,c-a);}
42    inline int orientation(PT a,PT b,PT c){return sign(cross(b-a,c-a))
43        ;}
44
45    PT perp(PT a){return PT(-a.y,a.x);}

```

```

43 PT rotateccw90(PT a){return PT(-a.y, a.x);}
44 PT rotatecw90(PT a){return PT(a.y,-a.x);}
45 PT rotateccw(PT a,double t){return PT(a.x*cos(t)-a.y*sin(t),a.x*
    sin(t)+a.y*cos(t));}
46 PT rotatecw(PT a,double t){return PT(a.x*cos(t)+a.y*sin(t),-a.x*
    sin(t)+a.y*cos(t));}

47 double SQ(double x){return x*x;}
48 double rad_to_deg(double r){return (r*180.0/PI);}
49 double deg_to_rad(double d){return (d*PI/180.0);}
50 double get_angle(PT a, PT b){
51     double costheta = dot(a,b)/a.norm()/b.norm();
52     return acos(max((double)-1.0, min((double)1.0, costheta)));
53 }
54
55 // does point p lie in angle <bac
56 bool is_point_in_angle(PT b,PT a,PT c,PT p){
57     assert(orientation(a,b,c)!= 0);
58     if(orientation(a,c,b)<0)swap(b,c);
59     return orientation(a,c,p)>=0 && orientation(a,b,p)<=0;
60 }
61
62 bool half(PT p) {
63     return p.y > 0.0 || (p.y == 0.0 && p.x < 0.0);
64 }
65
66 // sort points in counterclockwise
67 void polar_sort(vector<PT> &v){
68     sort(all(v), [](PT a,PT b){
69         return make_tuple(half(a),0.0,a.norm2())<make_tuple(half(b),
            cross(a,b),b.norm2());
70     });
71 }
72
73 //sort points in counterclockwise with respect to point o
74 void polar_sort(vector<PT> &v, PT o){
75     sort(all(v), [&](PT a,PT b){
76         return make_tuple(half(a-o),0.0,(a-o).norm2())<make_tuple(half
            (b-o),cross(a-o,b-o),(b-o).norm2());
77     });
78 }

```

10.2 Line

```

1 struct line{
2     PT a, b; // goes through points a and b
3     PT v; double c; //line form: direction vec [cross] (x, y) = c
4     line() {}
5
6     //direction vector v and offset c
7     line(PT v, double c) : v(v), c(c){
8         auto p = get_points();
9         a = p.first; b = p.second;
10    }
11
12    // equation ax + by + c = 0
13    line(double _a, double _b, double _c):v({_b, -_a}),c(-_c){
14        auto p = get_points();
15        a = p.first; b = p.second;
16    }
17
18    // goes through points p and q
19    line(PT p, PT q) : v(q - p), c(cross(v, p)), a(p), b(q) {}
20
21    pair<PT, PT> get_points(){//extract any two points from this
        line
22        PT p,q; double a = -v.y, b = v.x; // ax + by = c
23        if(sign(a) == 0){
24            p = PT(0, c / b);

```

```

25        q = PT(1, c / b);
26    }else if (sign(b) == 0){
27        p = PT(c/a,0);
28        q = PT(c/a,1);
29    }else{
30        p = PT(0, c/b);
31        q = PT(1, (c-a)/b);
32    }
33    return {p, q};
34 }
35
36 //ax + by + c = 0
37 array<double, 3> get_abc(){
38     double a = -v.y, b = v.x;
39     return {a, b, -c};
40 }
41
42 // 1 if on the left, -1 if on the right, 0 if on the line
43 int side(PT p){return sign(cross(v, p) - c);}
44
45 // line that is perpendicular to this and goes through point p
46 line perpendicular_through(PT p) {return {p, p + perp(v)};}
47 // translate the line by vector t i.e. shifting it by vector t
48 line translate(PT t) { return {v, c + cross(v, t)}; }
49 // compare two points by their orthogonal projection on this
    line
50 // a projection point comes before another if it comes first
    according to vector v
51 bool cmp_by_projection(PT p, PT q) { return dot(v, p) < dot(v, q
    ); }
52
53 line shift_left(double d) {
54     PT z = v.perp().truncate(d);
55     return line(a + z, b + z);
56 }
57
58 // find a point from a through b with distance d
59 PT point_along_line(PT a, PT b, double d){
60     assert(a != b);
61     return a + ((b - a) / (b - a).norm()) * d;
62 }
63
64 // projection point c onto line through a and b assuming a != b
65 PT project_from_point_to_line(PT a, PT b, PT c){
66     return a + (b - a) * dot(c - a, b - a)/(b - a).norm2();
67 }
68 // reflection point c onto line through a and b assuming a != b
69 PT reflection_from_point_to_line(PT a, PT b, PT c){
70     PT p = project_from_point_to_line(a,b,c);
71     return p + p - c;
72 }
73
74 // minimum distance from point c to line through a and b
75 double dist_from_point_to_line(PT a, PT b, PT c){
76     return fabs(cross(b - a, c - a)/(b - a).norm());
77 }
78
79 // returns true if point p is on line segment ab
80 bool is_point_on_seg(PT a, PT b, PT p){
81     if(fabs(cross(p - b, a - b)) < eps) {
82         if (p.x < min(a.x, b.x) - eps || p.x > max(a.x, b.x) + eps)
83             return false;
84         if (p.y < min(a.y, b.y) - eps || p.y > max(a.y, b.y) + eps)
85             return false;
86         return true;
87     }
88     return false;
89 }
90
91 // minimum distance point from point c to segment ab that lies on
    segment ab

```

```

86 PT project_from_point_to_seg(PT a, PT b, PT c){
87     double r = dist2(a, b);
88     if (sign(r) == 0) return a;
89     r = dot(c - a, b - a) / r;
90     if(r<0)return a;
91     if(r>1)return b;
92     return a + (b - a) * r;
93 }
94
95 // minimum distance from point c to segment ab
96 double dist_from_point_to_seg(PT a, PT b, PT c){
97     return dist(c, project_from_point_to_seg(a, b, c));
98 }
99
100 // 0 if not parallel, 1 if parallel, 2 if collinear
101 int is_parallel(PT a, PT b, PT c, PT d){
102     double k = fabs(cross(b - a, d - c));
103     if(k < eps){
104         if(fabs(cross(a - b, a - c)) < eps && fabs(cross(c - d, c - a)
            ) < eps) return 2;
105         else return 1;
106     }else return 0;
107 }
108
109 // check if two lines are same
110 bool are_lines_same(PT a, PT b, PT c, PT d){
111     if(fabs(cross(a - c, c - d)) < eps && fabs(cross(b - c, c - d))
        < eps) return true;
112     return false;
113 }
114
115 // bisector vector of <abc
116 PT angle_bisector(PT &a, PT &b, PT &c){
117     PT p = a - b, q = c - b;
118     return p + q * sqrt(dot(p, p) / dot(q, q));
119 }
120
121 // 1 if point is ccw to the line, 2 if point is cw to the line, 3
    if point is on the line
122 int point_line_relation(PT a, PT b, PT p){
123     int c = sign(cross(p - a, b - a));
124     if (c < 0) return 1;
125     if (c > 0) return 2;
126     return 3;
127 }
128
129 // intersection point between ab and cd assuming unique
    intersection exists
130 bool line_line_intersection(PT a, PT b, PT c, PT d, PT &ans){
131     double a1 = a.y - b.y, b1 = b.x - a.x, c1 = cross(a, b);
132     double a2 = c.y - d.y, b2 = d.x - c.x, c2 = cross(c, d);
133     double det = a1 * b2 - a2 * b1;
134     if(det == 0) return 0;
135     ans = PT((b1 * c2 - b2 * c1) / det, (c1 * a2 - a1 * c2) / det);
136     return 1;
137 }
138
139 // intersection point between segment ab and segment cd assuming
    unique intersection exists
140 bool seg_seg_intersection(PT a, PT b, PT c, PT d, PT &ans){
141     double oa = cross2(c, d, a), ob = cross2(c, d, b);
142     double oc = cross2(a, b, c), od = cross2(a, b, d);
143     if(oa * ob < 0 && oc * od < 0){
144         ans = (a * ob - b * oa) / (ob - oa);
145         return 1;
146     }
147     else return 0;
148 }
149
150 // intersection point between segment ab and segment cd assuming
    unique intersection may not exists
151 // se.size()==0 means no intersection
152 // se.size()==1 means one intersection
153 // se.size()==2 means range intersection
154 set<PT> seg_seg_intersection_inside(PT a, PT b, PT c, PT d){

```



```

147 PT ans;
148 if (seg_seg_intersection(a, b, c, d, ans)) return {ans};
149 set<PT> se;
150 if (is_point_on_seg(c, d, a)) se.insert(a);
151 if (is_point_on_seg(c, d, b)) se.insert(b);
152 if (is_point_on_seg(a, b, c)) se.insert(c);
153 if (is_point_on_seg(a, b, d)) se.insert(d);
154 return se;
155 }
156 // intersection between segment ab and line cd
157 // 0 if do not intersect, 1 if proper intersect, 2 if segment
    intersect
158 int seg_line_relation(PT a, PT b, PT c, PT d){
159     double p = cross2(c, d, a);
160     double q = cross2(c, d, b);
161     if(sign(p) == 0 && sign(q) == 0) return 2;
162     else if(p * q < 0) return 1;
163     else return 0;
164 }
165 // intersection between segment ab and line cd assuming unique
    intersection exists
166 bool seg_line_intersection(PT a, PT b, PT c, PT d, PT &ans){
167     bool k = seg_line_relation(a, b, c, d);
168     assert(k != 2);
169     if(k) line_line_intersection(a, b, c, d, ans);
170     return k;
171 }
172 // minimum distance from segment ab to segment cd
173 double dist_from_seg_to_seg(PT a, PT b, PT c, PT d){
174     PT dummy;
175     if(seg_seg_intersection(a, b, c, d, dummy)) return 0.0;
176     else return min({dist_from_point_to_seg(a, b, c),
        dist_from_point_to_seg(a, b, d),
        dist_from_point_to_seg(c, d, a), dist_from_point_to_seg(c,
        d, b)});
177 }
178 }
179 // minimum distance from point c to ray (starting point a and
    direction vector b)
180 double dist_from_point_to_ray(PT a, PT b, PT c){
181     b = a + b;
182     double r = dot(c - a, b - a);
183     if (r < 0.0) return dist(c, a);
184     return dist_from_point_to_line(a, b, c);
185 }
186 // starting point as and direction vector ad
187 bool ray_ray_intersection(PT as, PT ad, PT bs, PT bd){
188     double dx = bs.x - as.x, dy = bs.y - as.y;
189     double det = bd.x * ad.y - bd.y * ad.x;
190     if (fabs(det) < eps) return 0;
191     double u = (dy * bd.x - dx * bd.y) / det;
192     double v = (dy * ad.x - dx * ad.y) / det;
193     if(sign(u) >= 0 && sign(v) >= 0) return 1;
194     else return 0;
195 }
196 double ray_ray_distance(PT as, PT ad, PT bs, PT bd){
197     if(ray_ray_intersection(as, ad, bs, bd)) return 0.0;
198     double ans = dist_from_point_to_ray(as, ad, bs);
199     ans = min(ans, dist_from_point_to_ray(bs, bd, as));
200     return ans;
201 }

```

10.3 Circle

```

1 struct circle{
2     PT p; double r;
3     circle() {}
4     circle(PT _p, double _r): p(_p), r(_r) {};
5     // center (x, y) and radius r

```

```

6     circle(double x, double y, double _r): p(PT(x, y)), r(_r) {};
7     // circumcircle of a triangle
8     // the three points must be unique
9     circle(PT a, PT b, PT c){
10         b = (a + b) * 0.5;
11         c = (a + c) * 0.5;
12         line_line_intersection(b, b + rotatecw90(a - b), c, c +
            rotatecw90(a - c), p);
13         r = dist(a, p);
14     }
15     // inscribed circle of a triangle
16     // pass a bool just to differentiate from circumcircle
17     circle(PT a, PT b, PT c, bool t){
18         line u, v;
19         double m = atan2(b.y - a.y, b.x - a.x), n = atan2(c.y - a.y, c
            .x - a.x);
20         u.a = a;
21         u.b = u.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
22         v.a = b;
23         m = atan2(a.y - b.y, a.x - b.x), n = atan2(c.y - b.y, c.x - b
            .x);
24         v.b = v.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
25         line_line_intersection(u.a, u.b, v.a, v.b, p);
26         r = dist_from_point_to_seg(a, b, p);
27     }
28     bool operator == (circle v) { return p == v.p && sign(r - v.r)
        == 0; }
29     double area() { return PI * r * r; }
30     double circumference() { return 2.0 * PI * r; }
31 };
32
33 //0 if outside, 1 if on circumference, 2 if inside circle
34 int circle_point_relation(PT p, double r, PT b){
35     double d = dist(p, b);
36     if(sign(d - r) < 0) return 2;
37     if (sign(d - r) == 0) return 1;
38     return 0;
39 }
40 // 0 if outside, 1 if on circumference, 2 if inside circle
41 int circle_line_relation(PT p, double r, PT a, PT b){
42     double d = dist_from_point_to_line(a, b, p);
43     if (sign(d - r) < 0) return 2;
44     if (sign(d - r) == 0) return 1;
45     return 0;
46 }
47 //compute intersection of line through points a and b with
48 //circle centered at c with radius r > 0
49 vector<PT> circle_line_intersection(PT c, double r, PT a, PT b){
50     vector<PT> ret;
51     b = b - a; a = a - c;
52     double A = dot(b, b), B = dot(a, b);
53     double C = dot(a, a) - r * r, D = B * B - A * C;
54     if (D < -eps) return ret;
55     ret.push_back(c + a + b * (-B + sqrt(D + eps)) / A);
56     if (D > eps) ret.push_back(c + a + b * (-B - sqrt(D)) / A);
57     return ret;
58 }
59 //5 - outside and do not intersect
60 //4 - intersect outside in one point
61 //3 - intersect in 2 points
62 //2 - intersect inside in one point
63 //1 - inside and do not intersect
64 int circle_circle_relation(PT a, double r, PT b, double R){
65     double d = dist(a, b);
66     if (sign(d - r - R) > 0) return 5;
67     if (sign(d - r - R) == 0) return 4;
68     double l = fabs(r - R);

```

```

69     if (sign(d - r - R) < 0 && sign(d - l) > 0) return 3;
70     if (sign(d - l) == 0) return 2;
71     if (sign(d - l) < 0) return 1;
72     assert(0); return -1;
73 }
74 vector<PT> circle_circle_intersection(PT a, double r, PT b, double
    R){
75     if (a == b && sign(r - R) == 0) return {PT(1e18, 1e18)};
76     vector<PT> ret;
77     double d = sqrt(dist2(a, b));
78     if (d > r + R || d + min(r, R) < max(r, R)) return ret;
79     double x = (d * d - R * R + r * r) / (2 * d);
80     double y = sqrt(r * r - x * x);
81     PT v = (b - a) / d;
82     ret.push_back(a + v * x + rotateccw90(v) * y);
83     if (y > 0) ret.push_back(a + v * x - rotateccw90(v) * y);
84     return ret;
85 }
86 // returns two circle c1, c2 through points a, b and of radius r
87 // 0 if there is no such circle, 1 if one circle, 2 if two circle
88 int get_circle(PT a, PT b, double r, circle &c1, circle &c2){
89     vector<PT> v = circle_circle_intersection(a, r, b, r);
90     int t = v.size();
91     if(!t) return 0;
92     c1.p = v[0], c1.r = r;
93     if(t == 2) c2.p = v[1], c2.r = r;
94     return t;
95 }
96 // returns area of intersection between two circles
97 double circle_circle_area(PT a, double r1, PT b, double r2){
98     double d = (a - b).norm();
99     if(r1 + r2 < d + eps) return 0;
100     if(r1 + d < r2 + eps) return PI * r1 * r1;
101     if(r2 + d < r1 + eps) return PI * r2 * r2;
102     double theta_1 = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d)
        ),
103         theta_2 = acos((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d));
104     return r1 * r1 * (theta_1 - sin(2 * theta_1) / 2.) + r2 * r2 * (
        theta_2 - sin(2 * theta_2) / 2.);
105 }

```

10.4 Tangent

```

1 // tangent lines from point q to the circle
2 int tangent_lines_from_point(PT p, double r, PT q, line &u, line &
    v){
3     int x = sign(dist2(p, q) - r * r);
4     if (x < 0) return 0; // point in cricle
5     if (x == 0){ // point on circle
6         u = line(q, q + rotateccw90(q - p));
7         v = u;
8         return 1;
9     }
10
11     double d = dist(p, q);
12     double l = r * r / d;
13     double h = sqrt(r * r - l * l);
14     u = line(q, p + ((q - p).truncate(1) + (rotateccw90(q - p).
        truncate(h))));
15     v = line(q, p + ((q - p).truncate(1) + (rotatecw90(q - p).
        truncate(h))));
16     return 2;
17 }
18 // returns outer tangents line of two circles
19 // if inner == 1 it returns inner tangent lines
20 int tangents_lines_from_circle(PT c1, double r1, PT c2, double r2,
    bool inner, line &u, line &v){
21     if(inner) r2 = -r2;

```



```

22 PT d = c2 - c1;
23 double dr = r1 - r2, d2 = d.norm2(), h2 = d2 - dr * dr;
24 if(d2 == 0 || h2 < 0){
25     assert(h2 != 0);
26     return 0;
27 }
28 vector<pair<PT, PT>>out;
29 for (int tmp: {- 1, 1}){
30     PT v = (d * dr + rotateccw90(d) * sqrt(h2) * tmp) / d2;
31     out.push_back({c1 + v * r1, c2 + v * r2});
32 }
33 u = line(out[0].first, out[0].second);
34 if(out.size() == 2) v = line(out[1].first, out[1].second);
35 return 1 + (h2 > 0);
36 }
37 // returns two circle c1, c2 which is tangent to line u, goes
   through
38 // point q and has radius r1; 0 for no circle, 1 if c1 = c2 , 2 if
   c1 != c2
39 int get_circle(line u, PT q, double r1, circle &c1, circle &c2){
40     double d = dist_from_point_to_line(u.a, u.b, q);
41     if (sign(d - r1 * 2.0) > 0) return 0;
42     if (sign(d) == 0) {
43         cout << u.v.x << ' ' << u.v.y << '\n';
44         c1.p = q + rotateccw90(u.v).truncate(r1);
45         c2.p = q + rotatecw90(u.v).truncate(r1);
46         c1.r = c2.r = r1;
47         return 2;
48     }
49     line u1 = line(u.a + rotateccw90(u.v).truncate(r1), u.b +
   rotateccw90(u.v).truncate(r1));
50     line u2 = line(u.a + rotatecw90(u.v).truncate(r1), u.b +
   rotatecw90(u.v).truncate(r1));
51     circle cc = circle(q, r1);
52     PT p1, p2; vector<PT> v;
53     v = circle_line_intersection(q, r1, u1.a, u1.b);
54     if(!v.size()) v = circle_line_intersection(q, r1, u2.a, u2.b);
55     v.push_back(v[0]);
56     p1 = v[0], p2 = v[1];
57     c1 = circle(p1, r1);
58     if(p1 == p2){
59         c2 = c1;
60         return 1;
61     }
62     c2 = circle(p2, r1);
63     return 2;
64 }

```

10.5 Convex Hull

```

1 vector<PT> convex_hull(vector<PT> &p){
2     if (p.size() <= 1) return p;
3     vector<PT> v = p;
4     sort(v.begin(), v.end());
5     vector<PT> up, dn;
6     for(auto& p:v){
7         while(up.size() > 1 && orientation(up[up.size() - 2], up.back
   (), p) >= 0)
8             up.pop_back();
9
10        while(dn.size() > 1 && orientation(dn[dn.size() - 2], dn.back
   (), p) <= 0)
11            dn.pop_back();
12
13        up.push_back(p);
14        dn.push_back(p);
15    }
16    v = dn;

```

```

17     if (v.size() > 1) v.pop_back();
18     reverse(up.begin(), up.end());
19     up.pop_back();
20     for (auto& p : up)
21         v.push_back(p);
22
23     if (v.size() == 2 && v[0] == v[1]) v.pop_back();
24     return v;
25 }

```

10.6 Polygon

```

1 double area_of_triangle(PT a, PT b, PT c) {
2     return fabs(cross(b - a, c - a) * 0.5);
3 }
4 // -1 if strictly inside, 0 if on the polygon, 1 if strictly
   outside
5 int is_point_in_triangle(PT a, PT b, PT c, PT p) {
6     if (sign(cross(b - a, c - a)) < 0) swap(b, c);
7     int c1 = sign(cross(b - a, p - a));
8     int c2 = sign(cross(c - b, p - b));
9     int c3 = sign(cross(a - c, p - c));
10    if (c1<0 || c2<0 || c3 < 0) return 1;
11    if (c1 + c2 + c3 != 3) return 0;
12    return -1;
13 }
14 double perimeter(vector<PT> &p) {
15     double ans=0; int n = p.size();
16     for (int i = 0; i < n; i++) ans += dist(p[i], p[(i + 1) % n]);
17     return ans;
18 }
19 double area(vector<PT> &p) {
20     double ans = 0; int n = p.size();
21     for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1) % n])
   ;
22     return fabs(ans) * 0.5;
23 }

```

11 Miscellaneous

11.1 Rnd template

```

1 mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().
   count());
2 uniform_int_distribution<ll> dist(1ll, INF64);
3 // dist(rnd)/3 , rnd()/3

```